

ARM PrimeCell

General Purpose Input/Output (PL061)

Design Manual

ARM PrimeCell General Purpose Input/Output Design Manual

© Copyright ARM Limited 2000. All rights reserved.

Proprietary notice

ARM, the ARM powered logo, Thumb and StrongARM are registered trademarks of ARM Limited.

The ARM logo, PrimeCell, AMBA, Angel, ARMulator, EmbeddedICE, ModelGen, Multi-ICE, ARM7TDMI, ARM9TDMI, TDMI and STRONG are trademarks of ARM Limited.

All other products or services mentioned herein may be trademarks of their respective owners.

Neither the whole nor any part of the information contained in, or the product described in, this document may be adapted or reproduced in any material form except with the prior written permission of the copyright holder.

The product described in this document is subject to continuous developments and improvements. All particulars of the product and its use contained in this document are given by ARM Limited in good faith. However, all warranties implied or expressed, including but not limited to implied warranties or merchantability, or fitness for purpose, are excluded.

This document is intended only to assist the reader in the use of the product. ARM Limited shall not be liable for any loss or damage arising from the use of any information in this document, or any error or omission in such information, or any incorrect use of the product.

Document confidentiality status

This document is ARM Confidential (Distributed to ARM staff, product licensees & NDA signatories only).

Product status

The information in this document is Final (information on a developed product).

Web address

<http://www.arm.com>

Feedback

ARM Limited welcomes feedback on both the product, and the documentation.

Feedback on this document

If you have any comments on about this document, please send email to errata@arm.com giving:

- The document title
- The documents number
- The page number(s) to which your comments refer
- A concise explanation of your comments

General suggestion for additions and improvements are also welcome.

Feedback on the ARM PrimeCell General Purpose Input/Output

If you have any comments or suggestions about this product, please contact your supplier giving:

- The Product name
- A concise explanation of your comments.

Contents

1	ABOUT THIS DOCUMENT	1
1.1	Change control	1
1.1.1	Current status and anticipated changes	1
1.1.2	Change history	1
1.2	References	1
1.3	Terms and abbreviations	1
2	SCOPE	2
3	INTRODUCTION	2
4	GPIO DESIGN OVERVIEW	3
4.1	Signal Description	3
4.2	Programmer's Model	4
4.2.1	Functional Mode Registers	4
4.2.2	Integration Test Mode Registers	5
4.3	Block Partitioning	6
4.3.1	APB Interface and Register Block	7
4.3.2	Hardware Control Interface and Input/Output Multiplexor	8
4.3.3	Interrupt Detection Logic	9
5	GPIO IMPLEMENTATION DETAILS	10
5.1	Design Hierarchy	10
5.2	GPIO APB Interface (GpioApbif)	11
5.2.1	Write Interface	11
5.2.2	Read Interface	13
5.2.3	Control Registers	14
5.3	Interrupt Detection Logic Block (GPIOInt)	17
5.4	Hardware Control-APB Interface I/O Multiplexer (GPIOAfm)	18
5.5	Revision Designator block	19
5.6	Integration Test Harness	19
5.6.1	Intrachip Input Integration Test Harness	19
5.6.2	Primary inputs	20
5.6.3	Output Integration Test Harness	20

5.6.4	Primary Outputs	21
6	GPIO FUNCTIONAL VERIFICATION TEST DETAILS	22
6.1	GPIO VHDL Verification Testbench	22
6.1.1	Unit Under Test	24
6.1.2	APB Bridge Model	24
6.1.3	Trickbox	24
6.1.4	Protocol Checker	24
6.1.5	APB Multiplexer	24
6.1.6	Test Vectors	25
6.1.7	Parameters	25
6.1.8	Running Simulations	26
6.2	GPIO Verilog Verification Testbench	27
6.2.1	Unit Under Test	29
6.2.2	APB Bridge Model	29
6.2.3	Trickbox	29
6.2.4	Protocol Checker	29
6.2.5	APB Multiplexer	29
6.2.6	Test Vectors	30
6.2.7	Parameters	30
6.2.8	Running Simulations	31
6.3	Trickbox Description	32
6.3.1	Trickbox Signal Description	32
6.3.2	Trickbox functional description	33
6.4	Functional Tests	34
6.4.1	Reset State : RstTest.c	34
6.4.2	Register Access: TestGPIOReg.c	35
6.4.3	Register Access (GPIODATA Address Masking) : TestGPIODATAREgMasking.c	36
6.4.4	Interrupt Generation Capability : IntTests.c	36
6.4.5	Hardware Control Functionality : AltFunctTests.c	38
6.4.6	Integration Vectors Functionality: IntVectors.c	40
6.5	Verification Test Vector Generation	41
7	GPIO INTEGRATION TEST DETAILS	42
7.1	GPIO VHDL Integration testbench	42
7.1.1	Unit Under Test	44
7.1.2	Test Vectors	44
7.1.3	Running Simulations	44
7.2	GPIO Verilog Integration testbench	45
7.2.1	Unit Under Test	47
7.2.2	Test Vectors	47
7.2.3	Running Simulations	47
7.3	Integration Trickbox	48
7.3.1	Integration Trickbox Signal Description	48
7.3.2	Integration Trickbox Functional Description	48

7.4	Integration Tests	49
7.4.1	Pad signals nGPEN[7:0], GPOUT[7:0] and GPIN[7:0]	49
7.4.2	Integration Testing of intra-chip signals nGPAFEN[7:0], GPAFOUT[7:0] and GPAFIN[7:0]	50
7.4.3	Intra-chip outputs GPIOINTR and GPIOMIS[7:0]	50
7.5	Integration Test Vector Generation	51

1 ABOUT THIS DOCUMENT

1.1 Change control

1.1.1 Current status and anticipated changes

First release.

1.1.2 Change history

Issue	Date	Change
A01	1 September, 2000	First draft release for review
A02	4 September, 2000	Review comments implemented
A	12 September, 2000	First Release

1.2 References

This document refers to the following documents.

Ref.	Doc. No.	Title
1	ARM DDI 0190	ARM PrimeCell General Purpose Input/Output (PL061) Technical Reference Manual
2	PL061 INTM 0000 A	ARM PrimeCell General Purpose Input/Output (PL061) Integration Manual
3	ARM DUI 0006	AMBA Compliance Test Suite User Guide
4	ARM DDI 0138	Example AMBA System Technical Reference Manual
5	ARM DUI 0092	Example AMBA SYstem User Guide
7	SCB-QPRO-0008-A03	PrimeCell Integration Vector Strategy
8	SCB00-DESR-0001-A	AMBA Peripherals Design Manual
9	ARM-IHI-0011A	AMBA Specification (Rev 2.0)

1.3 Terms and abbreviations

This document uses the following terms and abbreviations.

Term	Meaning
AMBA	Advanced Micro-controller Bus Architecture
APB	Advanced Peripheral Bus
ASIC	Application Specific Integrated Circuit
HDL	Hardware Description Language

GPIO	General Purpose Input/Output
TIC	Test Interface Controller
VHDL	VHSIC Hardware Description Language
VHSIC	Very High Speed Integrated Circuit

2 SCOPE

This document describes the design and implementation of the ARM PrimeCell GPIO (General Purpose Input/Output). The design description is split between two sections – Section 4 and Section 5. The testbenches and the test programs for functional verification are described in Section 6.

Section 4 presents an overview of the GPIO design, describing the functional partitioning of the GPIO and the signal interconnections. Section 5 presents detailed descriptions on the GPIO design. Section 6 describes the testbenches and the test programs used to generate test vectors for functional verification of the GPIO. Section 7 describes the testbenches and the test programs used to generate test vectors for Integration Testing of the GPIO.

3 INTRODUCTION

The GPIO is a part of a library of reusable IP blocks, which can be more easily integrated into a typical ASIC design flow.

For further information on this block refer to the ARM PrimeCell General Purpose Input/Output (PL061) Technical Reference Manual [Ref.1].

4 GPIO DESIGN OVERVIEW

The PrimeCell GPIO is an interface between the APB system bus and eight programmable inputs or outputs that you can control in two modes:

- software mode through the APB bus interface
- hardware mode through a hardware control interface.

The CPU accesses data, control, and status information of the PrimeCell GPIO through the APB bus interface.

4.1 Signal Description

The following tables summarise the GPIO signal list. **Table 4.1-1** lists the APB interface signals, while **Table 4.1-2** lists the module-specific signals.

Name	Type	Source / Destination	Description
PCLK	In	Clock Generator	APB bus clock
PRESETn	In	Reset Controller	APB bus reset
PADDR[11:2]	In	APB Bridge	APB bus address
PSEL	In	APB Bridge	APB bus peripheral select
PENABLE	In	APB Bridge	APB bus enable
PWRITE	In	APB Bridge	APB bus write signal
PWDATA[7:0]	In	APB Bridge	APB bus write data
PRDATA[7:0]	Out	APB Bridge	APB bus read data

Table 4.1-1 APB interface signals

Name	Type	Source / Destination	Description
nGPEN[7:0]	Out	PAD	GPIO Output pad enable signal
GPOUT[7:0]	Out	PAD	GPIO Output data signal driver
GPIN[7:0]	In	PAD	GPIO Input data from pad
nGPAFEN[7:0]	In	Generic	Hardware control output enable
GPAFOUT[7:0]	In	Generic	Hardware control data input
GPAFIN[7:0]	Out	Generic	Hardware control data output
GPIOMIS[7:0]	Out	Interrupt Controller	Interrupt signals
GPIOINTR	Out	Interrupt Controller	Combined OR version of GPIOMIS
SCANENABLE	In	Test Controller	Scan enable
SCANINPCLK	In	PCLK scan chain	Scan data input signal for PCLK domain
SCANOUTPCLK	Out	PCLK scan chain	Scan data output for PCLK domain

Table 4.1-2 Module-specific signals

4.2 Programmer's Model

The GPIO registers are shown in **Table 4.2-1** and **Table 4.2-2**. The base address is implementation dependent. The registers located at offset 0x100, 0x104 and 0x108 are reserved for integration testing and are not used during normal operation. For a detailed description of the registers refer to the ARM PrimeCell General Purpose Input/Output (PL061) Technical Reference Manual [Ref. 1].

4.2.1 Functional Mode Registers

Address	Type	Width	Reset value	Name	Description
GPIO Base + 0x000 - 0x3FC	R/W	8	0x00	GPIODATA	GPIO Data Register
GPIO Base + 0x400	R/W	8	0x00	GPIODIR	GPIO Data Direction Register
GPIO Base + 0x404	R/W	8	0x00	GPIOIS	GPIO Interrupt Sense Register
GPIO Base + 0x408	R/W	8	0x00	GPIOIBE	GPIO Interrupt Both Edges Reg.
GPIO Base + 0x40C	R/W	8	0x00	GPIOIEV	GPIO Interrupt Event Register
GPIO Base + 0x410	R/W	8	0x00	GPIOIE	GPIO Interrupt Enable
GPIO Base + 0x414	R	8	0x00	GPIORIS	GPIO Raw Interrupt Status
GPIO Base + 0x418	R	8	0x00	GPIOMIS	GPIO Masked Interrupt Status
GPIO Base + 0x41C	W	8	0x00	GPIOIC	GPIO Interrupt Clear
GPIO Base + 0x420	R/W	8	0x00	GPIOAFSEL	GPIO Hardware Control Select
GPIO Base + 0x424 - 0xFCC	-	-	-	-	Reserved for future use and test purposes.
GPIO Base + 0xFD0 - 0xFDC	-	-	-	-	Reserved for future ID expansion.
GPIOBase+ 0xFE0	RO	8/0	0x80	GPIOPeriphID0	Peripheral ID register, bits 7:0
GPIOBase+ 0xFE4	RO	8/0	0x11	GPIOPeriphID1	Peripheral ID register, bits 15:8
GPIOBase+ 0xFE8	RO	8/0	0x04	GPIOPeriphID2	Peripheral ID register, bits 23:16
GPIOBase+ 0xFEC	RO	8/0	0x00	GPIOPeriphID3	Peripheral ID register, bits 31:24
GPIOBase+ 0xFF0	RO	8/0	0x0D	GPIOPCellID0	PrimeCell ID register, bits 7:0
GPIOBase+ 0xFF4	RO	8/0	0xF0	GPIOPCellID1	PrimeCell ID register, bits 15:8
GPIOBase+ 0xFF8	RO	8/0	0x05	GPIOPCellID2	PrimeCell ID register, bits 23:16
GPIOBase+ 0xFFC	RO	8/0	0xB1	GPIOPCellID3	PrimeCell ID register, bits 31:24

Table 4.2-1 Functional Mode Registers

4.2.2 Integration Test Mode Registers

Address	Type	Width	Reset value	Name	Description
GPIO Base + 0x600	R/W	1	0x0	GPIOITCR	Integration test control register
GPIO Base + 0x604	R/W	8	0x00	GPIOITIP1	Integration test input read/set Reg.
GPIO Base + 0x608	R/W	8	0x00	GPIOITIP2	Integration test input read/set Reg.
GPIO Base + 0x60C	W	8	0x00	GPIOITOP1	Integration test output set Reg.
GPIO Base + 0x610	R	1	0x0	GPIOITOP2	Integration test output set Reg.
GPIO Base + 0x614	R/W	8	0x00	GPIOITOP3	Integration test output set Reg.

Table 4.2-2 Integration Test Mode Registers

4.3 Block Partitioning

The GPIO Controller block diagram is shown in **Figure 4.3-1**. The design contains the following major sub-blocks.:

- APB Interface, Register Block
- Hardware Control Interface and Input/Output Multiplexor
- Interrupt Detection Logic

The design of these sub-blocks is detailed in the subsequent sections.

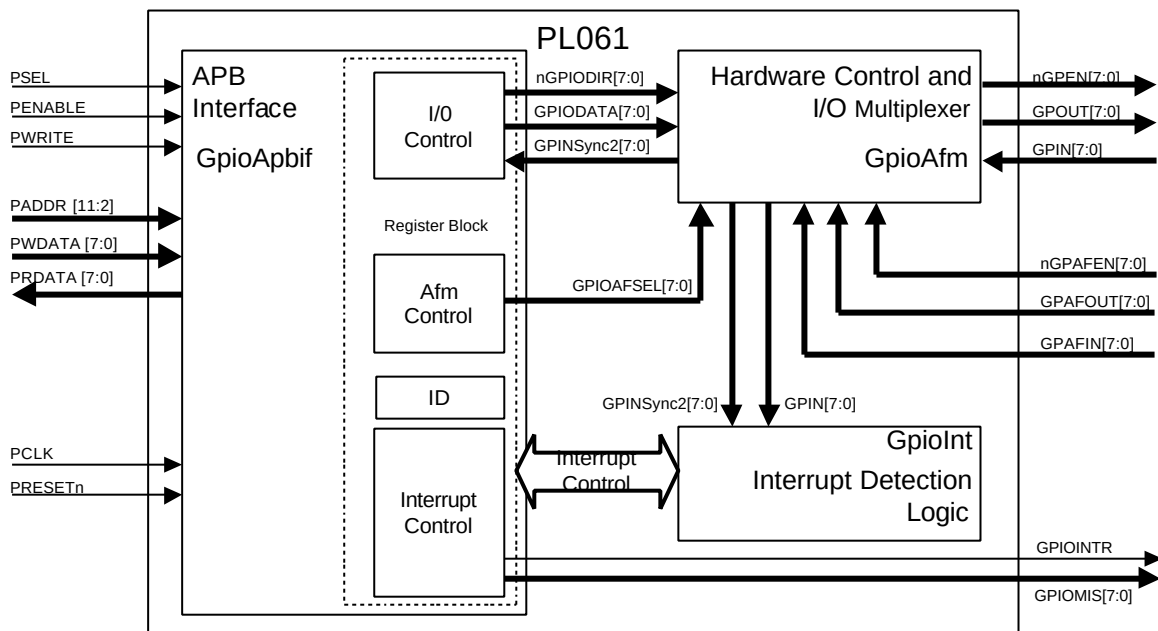


Figure 4.3-1 GPIO Block diagram

4.3.1 APB Interface and Register Block

This block decodes APB accesses and generates the write strobes to the appropriate registers. This block also contains all the writable registers in the GPIO, the output data multiplexer and the PRDATA output register. The internal databus for the GPIO, PWDATAIn[7:0], is also generated in this block, by gating the input data bus, PWDATA[7:0] with PSEL and PWRITE. The writable registers in the GPIO are GPIODATA, GPIODIR, GPIOIS, GPIOIBE, GPIOIEV, GPIOIE and GPIOAFSEL. GPIOIC is a virtual register without any storage elements.

This block has nine Interrupt Output signals available based on the Raw Interrupt Status, GPIORIS, register:

- GPIOINTR
- GPIOMIS[7:0]

GPIORIS and GPIOMIS are read-only virtual registers based on the Interrupt Detection Logic block functionality, refer to Section 5.3 .

A read-only decode is provided to access the PrimeCell and Peripheral Identification Registers.

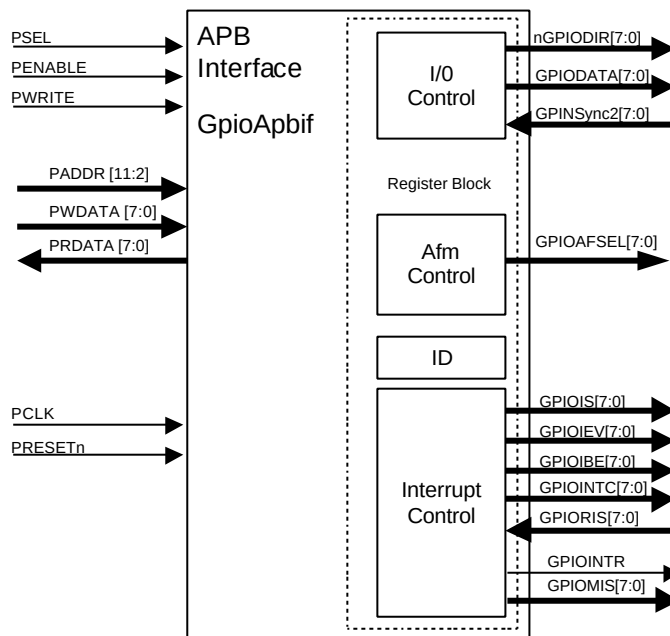


Figure 4.3-2 GPIO Block diagram

Integration Testing of the GPIO is supported by the GPIOITCR, GPIOITIP1, GPIOITIP2, GPIOITOP1, GPIOITOP2 and GPIOITOP3 registers, refer to Section 5.3.

4.3.2 Hardware Control Interface and Input/Output Multiplexor

The Hardware Control Interface selects whether the control over the GPIO lines **nGPEN**, **GPOUT** and **GPIN** is given to the APB Interface or to some external control block through the Hardware Control lines **nGPAFEN**, **GPAFOUT** and **GPIN**. **Figure 4.3-3** shows the block diagram of the GPIO Hardware Control and Input/Output Multiplexor.

Signals entering the GPIO through the GPIN lines are synchronized with flip-flops operating on the rising edge of PCLK before being sent to the APB interface and the Edge-Detection circuitry of the Interrupt Detection Logic block.

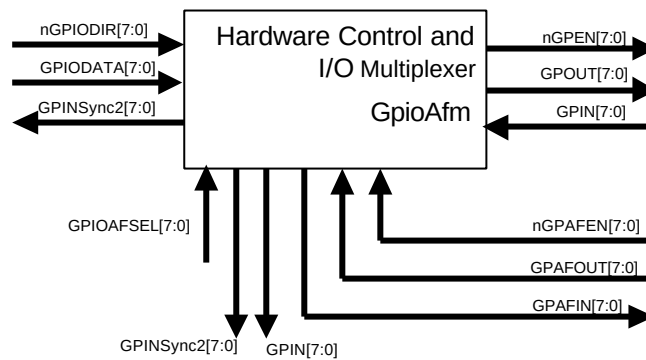


Figure 4.3-3 GPIO Hardware Control and Input/Output Multiplexor

This block contains part of the hardware required to support Integration testing of the GPIO. In essence, the test logic consists of multiplexors that switch between normal-mode I/O signals and test mode signals. The test mode stimuli can be controlled through the APB interface of the GPIO.

4.3.3 Interrupt Detection Logic

A simplified block diagram of the Interrupt Detection Logic is shown in **Figure 4.3-4**. Note that there are two versions of the signals appearing on the Pads entering this block: **GPIN** and **GPINSync2**. **GPIN** signals are, potentially, generated in an arbitrary clock domain, thus not synchronised to the **PCLK** domain when they are used for interrupt generation. Level detection is used on the **GPIN[7:0]** lines to allow interrupts to be set even in the absence of **PCLK**.

The interrupt logic generates a Raw Interrupt Status signal, **GPIORIS[7:0]**, which is fed into the APB Interface. The GPIOIE (Interrupt Enable) register in the APB interface controls whether to allow the Raw Interrupt Status to trigger an interrupt or to mask them on a bit by bit basis.

A third possible source of Data to analyse when searching for interrupt triggering events, is the APB-Interface controlled content of **GPIODATA**. These signals do not need to be synchronized when edge-conditions are expected. Therefore a dedicated line, **GPIODATA[7:0]**, is brought directly from the APB-Interface into the edge-detection circuitry of this block. For a detailed description of the Interrupt Detection Logic circuitry, refer to Section 5.3.

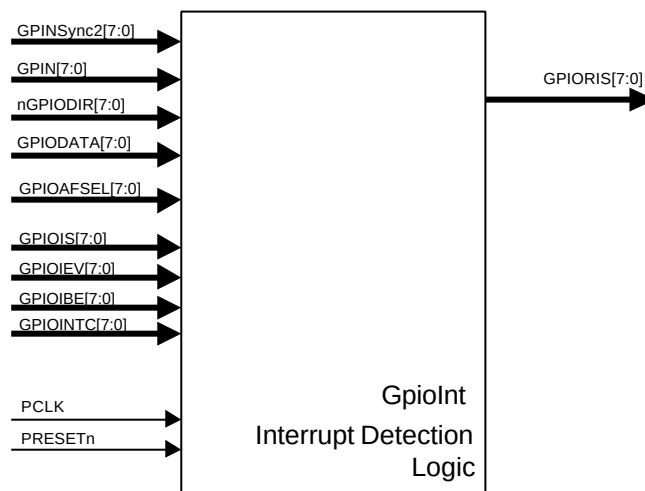


Figure 4.3-4 GPIO Interrupt Generation

5 GPIO IMPLEMENTATION DETAILS

5.1 Design Hierarchy

The hierarchy in the GPIO is shown in **Figure 5.1-1**. A short description about the functionality of each of the sub-modules is also given within brackets.

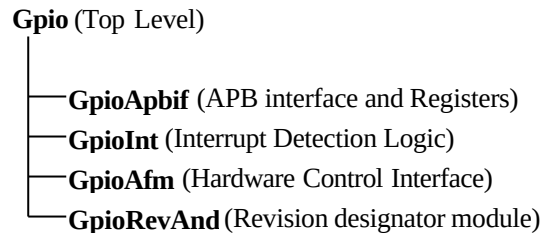


Figure 5.1-1 GPIO Design Hierarchy

5.2 GPIO APB Interface (GpioApbif)

The APB interface generates decodes for reads/writes into the registers in the address space of the GPIO. An internal write data bus, **PWDATAIn[7:0]**, is generated by gating the write data bus input to the GPIO, **PWDATA[7:0]** with **PSEL** and **PWRITE**. This prevents the data bus internal to the GPIO from toggling when the device is not selected and being written to. It is comprised of 3 separate functions Write Interface, Read Interface and Control Registers.

5.2.1 Write Interface

The write strobe for each register is generated with a decode of **PSEL**, **PWRITE**, **PADDR[11:2]** and **PENABLE**

Figure 5.2-1 below illustrates the generation of a write strobe (**GPIODIRwr**) for writes into the **GPIODIR** register and the latching of data into the register. Data is clocked into GPIODIR on the rising edge of **PCLK** when the Write enable (**GPIODIRwr**) is asserted.

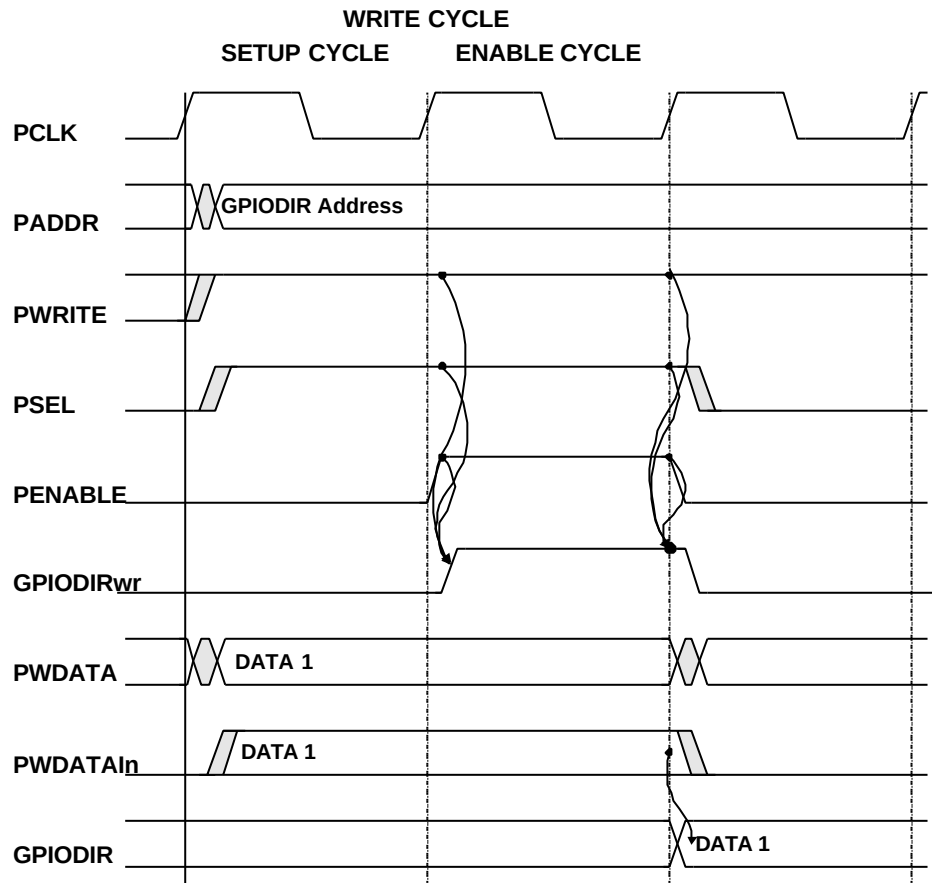


Figure 5.2-1 APB Write Access example

5.2.1.1 GPIODATA Write Address Masking

The data register GPIODATA can be addressed in 256 memory locations. This gives the user individual control of the GPIO line via the relevant address bit. The write strobe **GPIODATAwr** is gated with address lines **PADDR[9:2]** as shown in **Figure 5.2-2**.

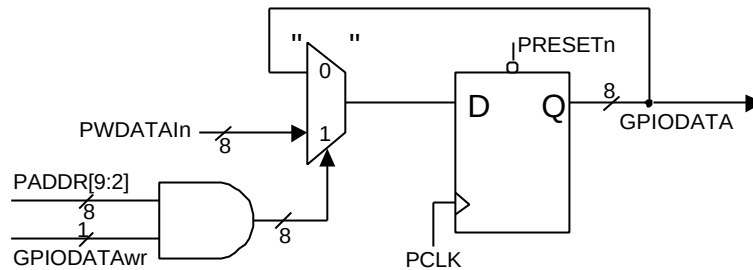


Figure 5.2-2: Write address-masking Logic.

Hence, only bit positions where the corresponding address line is a “1” are allowed to load a new value. Address lines holding a “0” cause the register to keep its previous value. See **Figure 5.2-3** for a detailed diagram.

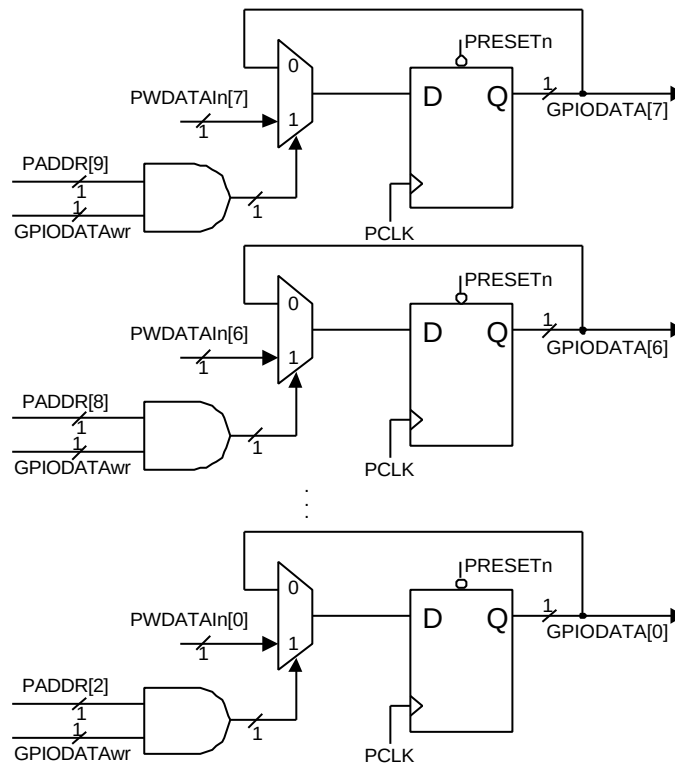


Figure 5.2-3: Detailed write-address-masking logic diagram

5.2.2 Read Interface

The read interface involves an output multiplexer followed by a bank of flip-flops, which drive data onto the data bus. These flip-flops are clocked by the rising edge of **PCLK**. Select inputs to the output multiplexer are generated with a combinational decode of **PWRITE**, **PADDR** and **PSEL** signals. Since these signals are stable from the rising edge of **PCLK**, almost one full **PCLK** period is available for the output of the multiplexer to stabilise before data is clocked in the next rising edge of **PCLK**.

The GPIO is an APB slave, which connects its read data bus **PRDATA** to the APB Bridge Multiplexor. This module is a simple multiplexor, with the read data buses from all peripheral bus slaves as the inputs, using the slave select bridge outputs as the select inputs, with a single read data bus as the output to the bridge module [Reference 4].

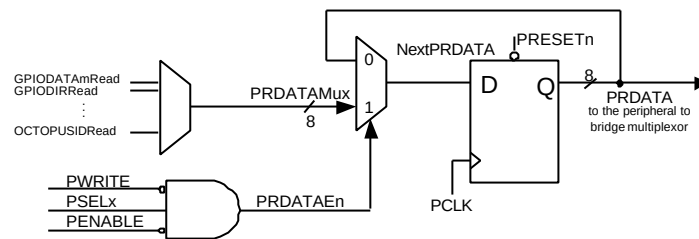


Figure 5.2-4: GPIO Registers to PRDATA bus interface

For read transfers the data can be driven on to the data bus when **PWRITE** is LOW and both **PSEL**(Gpio) and **PENABLE** are HIGH [Reference 9, 5.5.2], see **Figure 5.2-4:GPIO Registers to PRDATA bus interface**. Data is sampled by the APB Bridge on the next rising edge of **PCLK** on which **PENABLE** is deasserted.

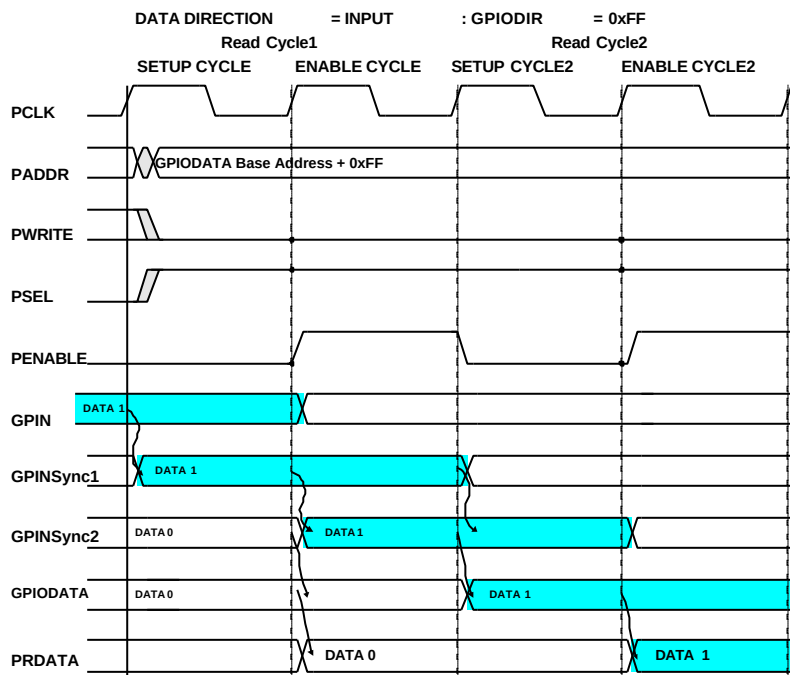


Figure 5.2-5: APB Read Access example

In order to reduce power consumption the data signal will not change after the transfer until the next transfer occurs, see **Figure 5.2-5**.

5.2.2.1 GPIODATA Read Address Masking

The values read from this register, **GPIODATA**, are determined for each bit, by the mask bit derived from the address used to access the data register.

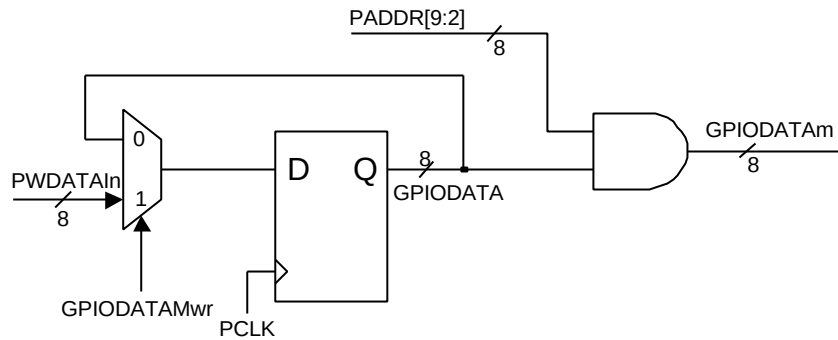


Figure 5.2-6: Write address-masking Logic

As it can be seen from **Figure 5.2-6**, address lines **PADDR[9:2]** being “0” do not allow the corresponding bits in GPIODATA to pass through the AND gate. Hence, masked bits of GPIODATA will be driven onto **GPIODATAm** lines as zero.

5.2.3 Control Registers

The control registers implement the storage elements for the I/O Control Logic, Hardware Control and interrupt generation.

5.2.3.1 I/O Control Logic

The GPIO has data and data direction registers. Reads from the data register give the current status of the corresponding I/O pins, whether they are configured as outputs or inputs. Writes to the data registers only affect the pins that are configured as outputs.

Note that writes/reads to the data register are address masked. See [Reference 1, pages 2-8 and 2-9] for further information on this function.

The contents of the data direction register determine whether the corresponding pins are configured as inputs or outputs. A “0” in the data direction register GPIODIR configures the corresponding pin as an input and a “1” configures the corresponding pin as an output.

5.2.3.2 Interrupt Control Registers

Interrupt functionality will be controlled via seven registers.

Three separate groups of registers can be identified. The first group of registers is used to configure the characteristics of the triggering source (rising edge, low level...). These are:

- GPIOIS
- GPIOIBE
- GPIOIEV

and are implemented as shown in **Figure 5.2-7**.

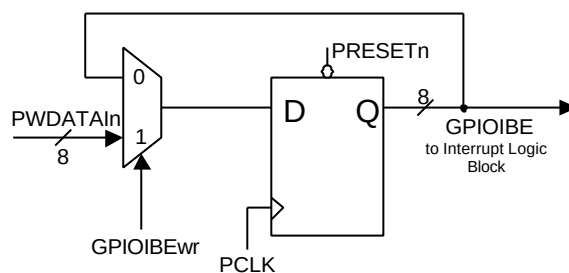


Figure 5.2-7: Example of register implementation, GPIOIBE

The second group of registers allows the software to clear the interrupt (edge case only), in order to enable the circuit for a possible new interrupt:

- GPIOIC

This register is implemented as write only (WO).

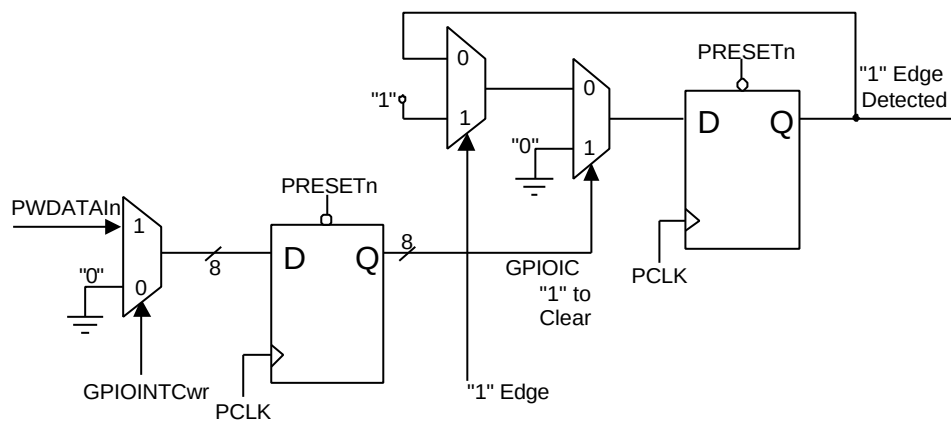


Figure 5.2-8: GPIOIC (WO) clears the Edge-Detected loop

There is no additional value in keeping the contents of this register. Therefore once the write operation finishes, the active-high bit-content of GPIOIC is kept for only one cycle of **PCLK**, being overwritten with zeroes afterwards. This is enough time for clearing the Edge-Detected loop in the Interrupt Detection Block. See **Figure 5.2-8**.

The final group of registers implement:

- GPIORIS
- GPIOIE
- GPIOMIS

See **Figure 5.2-9**. Note that GPIORIS is registered in the Interrupt Detection Logic block.

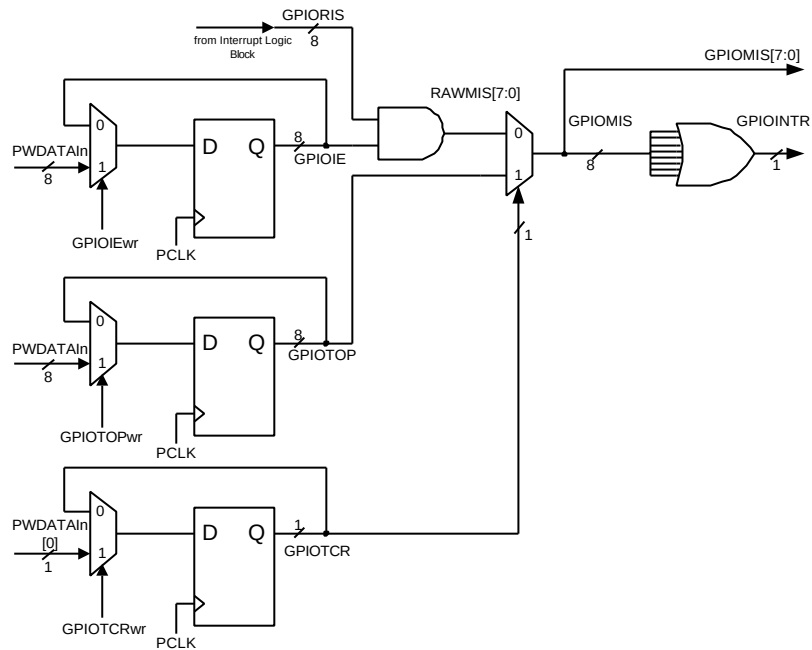


Figure 5.2-9: Mask Interrupt capabilities in the Interrupt Register Block

GPIORIS and GPIOMIS are read-only registers; they can only be modified from the Interrupt Detection Logic. The output **GPIOINTR** is an OR function of all the bits in GPIOMIS.

5.2.3.3 Hardware Control Select Registers

GPIO lines can be used to provide Hardware Control required for different system configurations by setting corresponding bits to '1' in the Hardware Control Function Select register. See **Figure 5.2-10**.

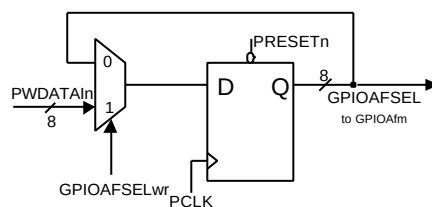


Figure 5.2-10: Hardware Control Register

A "1" in GPIOAFSEL indicates that Hardware Control lines are to be connected to the I/O pins. A "0" indicates that APB Interface functionality is driving the I/O pins.

5.3 Interrupt Detection Logic Block (GPIOInt)

The Interrupt Detection Logic has been designed to work on:

1. Data driven directly from the APB interface, through **GPIODATA[7:0]**.
2. Data present on the pads(potentially asynchronous), independently of the source driving them:
 - Pads, as inputs, connected to the APB interface.
 - Pads, as inputs, connected to the Hardware Control lines.
 - Pads, as outputs, connected to the Hardware Control lines.

Figure 5.3-1 shows the logic able to perform the required functionality.

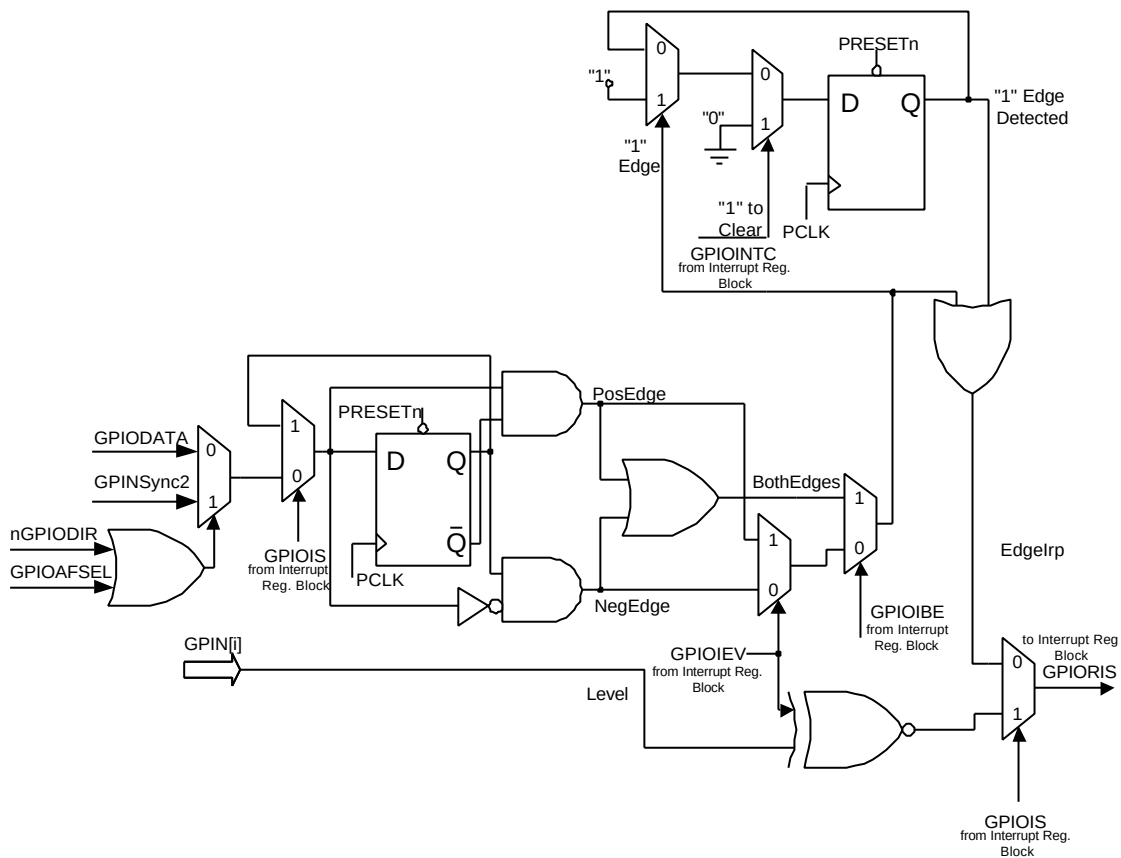


Figure 5.3-1: Interrupt Detection Logic slice diagram

As can be seen from the figure, the user can select, for any source, its polarity and edge properties. The module signals the existence of triggering conditions setting high the corresponding bits in GPIORIS.

GPIN can receive asynchronous data, this is synchronised before use in edge detection logic.

The edge detection logic is disabled when the relevant GPIOIS bit is '1', thereby reducing power consumption.

When the GPIO pins are configured to be driven by the APB interface, the interrupt latency is reduced by means of a direct usage of these synchronized signals into the interrupt detection logic.

5.5 Revision Designator block

This module contains a single AND gate to be used as a place-holder cell to mark the Revision of the block. The 2 input pins are tied-off at the top level of the hierarchy. These "TieOffs" can be identified during layout and re-wired to "VDD" or "VSS" if needed.

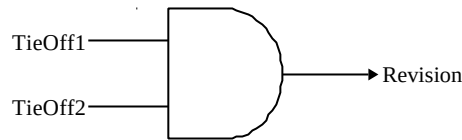


Figure 5.5-1 RevisionAnd component

This AND gate is instantiated four times at the top level, since the revision number is a four bit quantity. In the present version of the GPIO, all the inputs are tied to zero at the top level. Outputs of all AND gates are concatenated and connected to the output read multiplexer of the APB interface logic. This helps software to identify the revision number through an APB read.

5.6 Integration Test Harness

The GPIO integration test vector suite allows the integrator to verify that the GPIO has been connected into the target system correctly. The ITEN bit in the GPIOITCR register is used for configuring the GPIO in integration test mode. ITEN is low for normal operation. When ITEN is set high by software, the GPIO switches the multiplexors on the inputs and outputs to use the test mode values.

5.6.1 Intrachip Input Integration Test Harness

Figure 5.8-1 explains the implementation details of the input integration test harness. GPIOITIP1 and GPIOITIP2 are 8 bit write/read registers. The ITEN bit is used as the control bit for the multiplexer, which is used in the read path of the GPAFOUT and nGPAFEN intra-chip inputs. If the ITEN control bit is de-asserted, the GPAFOUT and nGPAFEN intra-chip inputs are routed as the internal GPAFOUT and nGPAFEN input signals. If the ITEN control bit is asserted the contents of GPIOITIP1 and GPIOITIP2 are driven on to the respective internal lines.

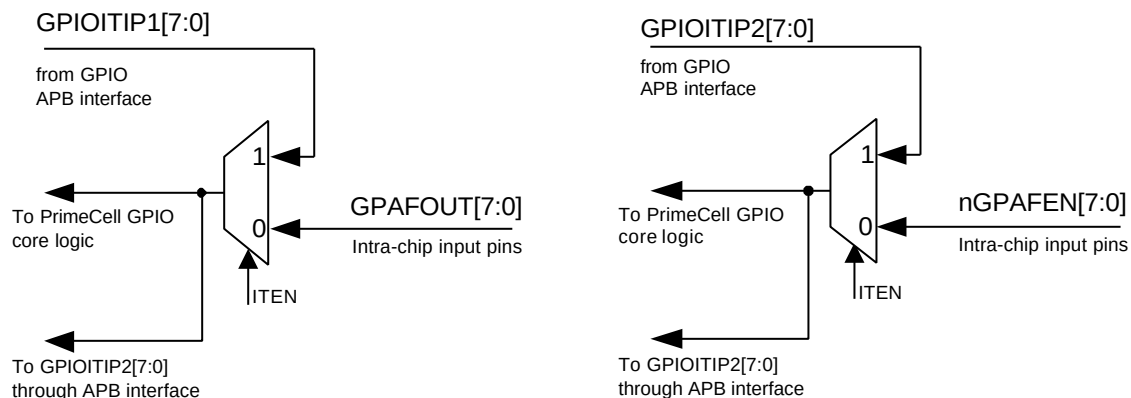


Figure 5.6-1 Input Integration test harness

5.6.2 Primary inputs

The primary inputs are tested using the integration vector trickbox by looping back primary input **GPIN[7:0]** as XOR **nGPEN[7:0]** and **GPOUT[7:0]** pairs.

5.6.3 Output Integration Test Harness

Figure 5.6-2 illustrates the implementation details of the output integration test harness in the case of intra-chip output **GPAFIN[7:0]**. **Figure 5.6-3** explains the implementation details of the output test integration register in case of Interrupt outputs **GPIOMIS[7:0]** and **GPIOINTR**. GPIOITOP1 allows setting the value of each **GPIOMIS** output bit and thus **GPIOINTR**.

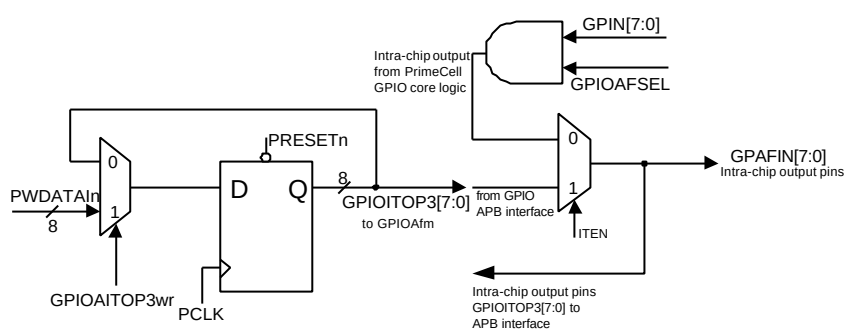


Figure 5.6-2 integration test harness Intra-chip output

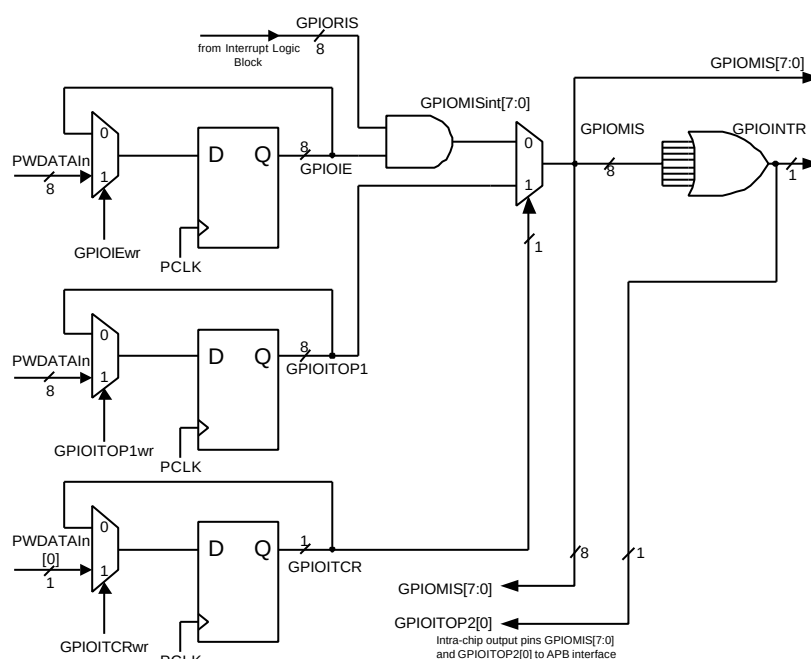


Figure 5.6-3 Integration test harness Interrupt Intra-chip output

5.6.4 Primary Outputs

Figure 5.6-4 explains the implementation details of the output test integration register in case of primary outputs **GPOUT[7:0]** and **nGPEN[7:0]**.

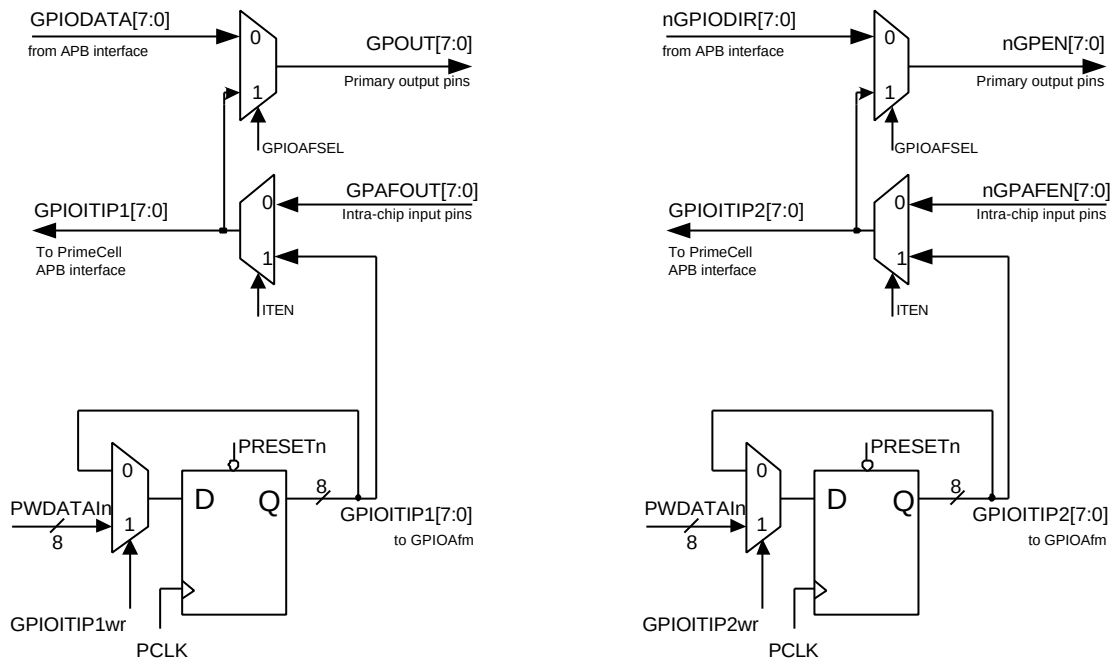


Figure 5.6-4 Output integration test harness Primary-outputs

Primary outputs **GPOUT[7:0]** and **nGPEN[7:0]** can be accessed through the **GPIOITIP1[7:0]** and **GPIOITIP2[7:0]** registers respectively by selecting integration test enable and hardware control mode.

6 GPIO FUNCTIONAL VERIFICATION TEST DETAILS

The GPIO Functional Verification testbench is an APB BusTalk testbench. The VHDL and Verilog BusTalk testbench files reside in the **verification/vhdl** and **verification/verilog** directories respectively. The test vectors that are applied to the BusTalk testbenches are in the **verification/bustest** directory.

6.1 GPIO VHDL Verification Testbench

The ARM PrimeCell GPIO VHDL test world uses a variant of a top level testbench which is based on a bus compliance testbench from the AMBA Compliance Test Suite. For further information refer to the ARM Micropack AMBA Compliance Test Suite User Guide.

The testbench is designed to be used with most common VHDL simulators, and it is suited to AMBA system designers who want to ensure the portability of their blocks to any other AMBA designs. This simplifies the ASIC integration task and exchange of peripherals between projects.

The testbench is “programmable”, meaning that a description of the transfers to be performed on the **Unit Under Test (UUT)** is given without a need to modify the test bench implementation, hence the “generic” property. This transfer description takes the form of a text file that can be read by the specific test bench to which it is being targeted.

Figure 6.1-1 illustrates the Bustalk VHDL testbench for running test vectors.

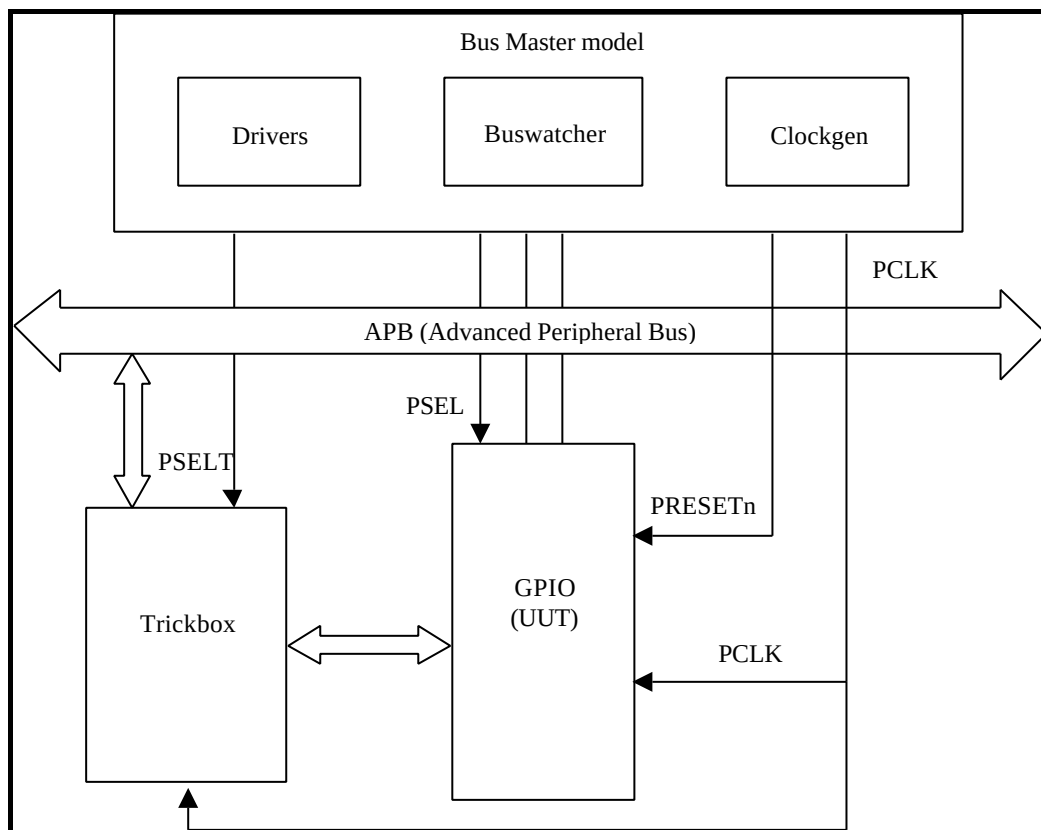


Figure 6.1-1 VHDL testbench for simulating test vectors

Figure 6.1-2 illustrates the hierarchy for the VHDL testbench.

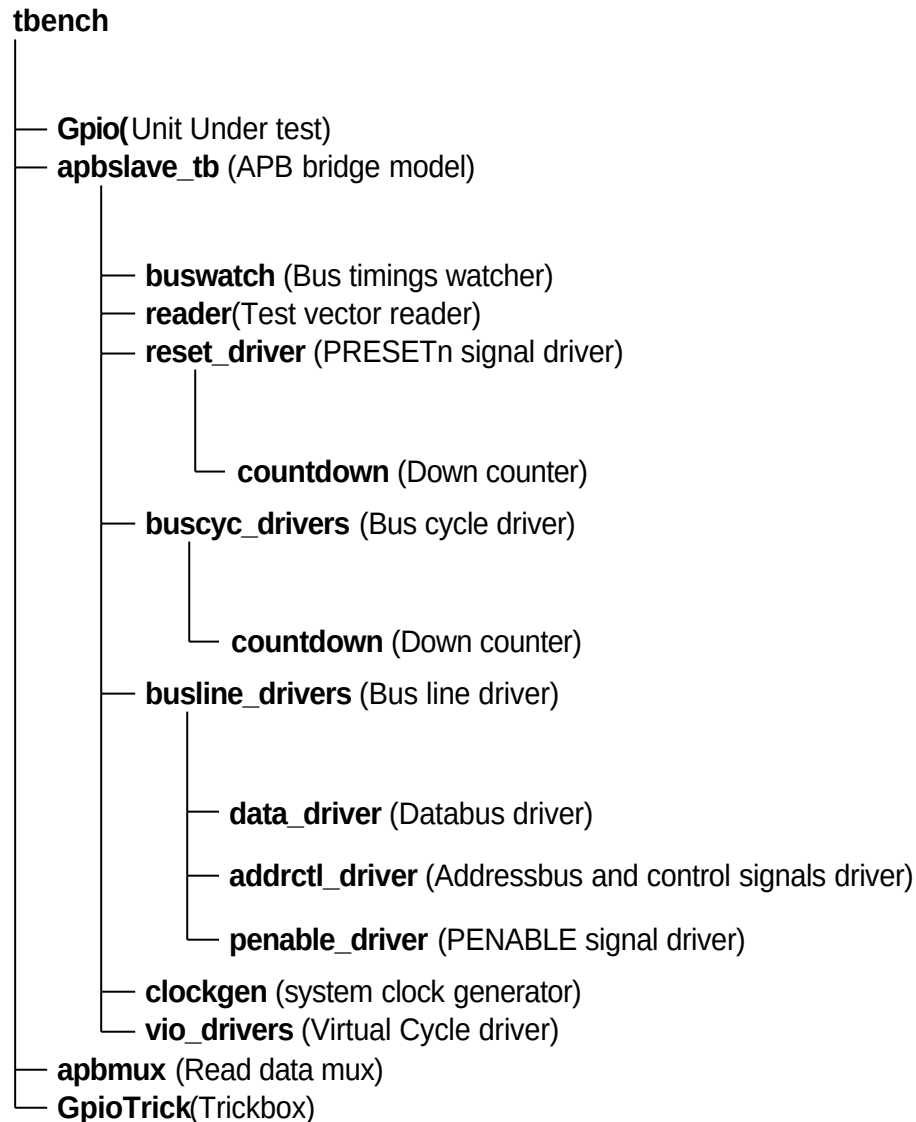


Figure 6.1-2 VHDL Testbench hierarchy

The testbench, **tbench.vhd** is used for functional simulations of the GPIO. This module instantiates the GPIO (Unit Under Test), the behavioural model of the APB Bridge, the trickbox and the APB Read mux.

The default test frequency is set at 100MHz. This is the same frequency to which the GPIO was constrained during synthesis. **PCLK** is set to the frequency defined in **tbench.vhd** using the **tlkh** and **tlkl** generic values when instantiating **apbslave_tb.vhd**.

6.1.1 Unit Under Test

The Unit Under Test may be one of the following

- GPIO VHDL RTL
- GPIO VHDL netlist from VHDL RTL Source

One out of these two versions may be referenced via the corresponding link to uut directory in **verification/vhdl** .

6.1.2 APB Bridge Model

The UUT is stimulated primarily by the APB bridge behavioural model. This module issues commands on the APB bus under program control.

The APB bridge model, `apbslave_tb.vhd`, instantiates the buswatcher, the reader, the reset driver, the bus cycle drivers, the bus line drivers and the clock generator. The reader module reads the test vectors from the `infile.bif` file (from the **verification/bustest/invec** directory) one line at a time and drives the information about the bus cycle in the form of packets. If the information driven by the reader indicates that the current cycle is a reset cycle, the reset driver drives the **PRESETn** signal with the correct timings. The `buscyc_drivers` instantiates an APB cycle driver for each cycle type, i.e PSW, PSR, PI, PO, etc. The `busline_drivers` drives the databus, the address and control signals with the correct timings. The buswatch module checks the Read/Write Databus timings. The clockgen generates the system clock, **PCLK**.

The APB bridge model also provides the Select line for the trickbox.

Any access to the address range

0x60 00 00 00 to 0x60 00 00 FF

will access the TrickBox via the **PSELT** select line.

Accesses to any address outside the above range will select the UUT via the **PSEL** select line.

6.1.3 Trickbox

The functionality of the GPIO is verified via a trickbox. The trickbox is an APB Slave which is designed to supply test stimulus to the non-AMBA input pins of the GPIO and capture the responses from the non-AMBA output pins of the same.

6.1.4 Protocol Checker

The testbench includes a protocol checker which reports any protocol or timing violations during accesses to any APB slave. Timing parameters can be set using the `timings.vhd` file in the **verification/vhdl/tbench** directory. The buswatch module which is in the **verification/vhdl/buswatcher** directory checks the Read/Write Databus timings and reports any violations in the timings.

6.1.5 APB Multiplexer

The read data buses from the GPIO and the trickbox are multiplexed together in this module before being connected to the APB Bridge model.

6.1.6 Test Vectors

The test vectors for the verification of the GPIO are coded into separate C files depending on the logical region of the GPIO, which these vectors are testing. Make options have been provided to use the entire set of test vectors together or to use them individually.

6.1.7 Parameters

The GPIO testbench provides some configurability options using generics and constants. These generics and constants are configurable through the `tbench.vhd` file.

XonPSEL

XonPSEL is used in the `tbench.vhd` file that resides in ***verification/vhdl/tbench***.

This generic is used to eliminate the 'X's that appear on the PSEL, PWRITE and PADDR lines when these lines are not being actively driven. If **XonPSEL** is set to **0** in the top level of the testbench, these signals will go to '0' when the corresponding line driver is inactive. If set to **1**, the signals go to 'X' when the line driver is idle.

Verbosity

Verbosity is used in the `data_driver.vhd`, the `reset_driver.vhd` and the `buscyc_drivers.vhd` which reside in ***verification/vhdl/bid***.

This generic is used to control the verbosity of the transcript generated during simulation. If **Verbosity** is set to **1**, every command issued will have a corresponding message in the transcript file. Every poll operation in a POLL command sequence is also reported. If **Verbosity** is set to **0**, a message is issued only if an error occurs on a read. Similarly during a POLL command, a message is flagged only if a POLL time-out occurs.

HaltOnMismatch

This is used in the `data_driver.vhd` which resides in ***verification/vhdl/bid***.

This generic is used to stop the simulation if a read is unsuccessful. If set to **1**, the simulation halts after an error is reported. If set to **0**, the error is flagged, but the simulation continues.

SuppressOnReset

SuppressOnReset is used in the `buswatch.vhd` which resides in ***verification/vhdl/buswatcher***.

This generic is used to suppress the checking of the **PRDATA** timings when the **PRESETn** signal is asserted. If **SuppressOnReset** is TRUE, the **PRDATA** set-up and hold timing violations are not flagged when **PRESETn** is asserted. If **SuppressOnReset** is set to FALSE, **PRDATA** set-up and hold timing violations are flagged irrespective of **PRESETn**.

Mesg_enb

Mesg_enb is used in the `buswatch.vhd` which resides in ***verification/vhdl/buswatcher***.

This generic is used to enable or disable the flagging of error messages when **PRDATA** does not go to zero when the UUT is not selected. The flagging of these error messages is disabled when this generic is set to FALSE. If **Mesg_enb** is set to TRUE, the error messages are flagged whenever **PRDATA** goes to a non-zero value when the UUT is not selected. This generic will have to be set to FALSE in testbenches which include more than one slave.

Data_check

Data_check is used in the `apbmux.vhd` which resides in ***verification/vhdl/tbench***.

This constant is used to enable or disable the flagging of error messages when the **PRDATA** driven by a slave has a non-zero value when it is not selected. When **Data_check** has a value **1**, these error messages are flagged.

When it has a value **0**, the error messages are not flagged. This constant is used in testbenches which include more than one slave.

6.1.8 Running Simulations

The user's guide gives step by step instructions for running simulations using the GPIO test vectors on the VHDL source code and the VHDL netlist.

Run time logs for the RTL simulations are:

- **verification/vhdl/Gpio_rtl/Gpio_rtl_modelsim.log**

Run time logs for the netlist simulations are:

- **verification/vhdl/Gpio_scaninsert_vhdl_net_max/Gpio_scaninsert_vhdl_net_max_modelsim.log**

The code coverage results are:

- **verification/vhdl/Gpio_rtl/vnavigator.summary**

6.2 GPIO Verilog Verification Testbench

The ARM PrimeCell GPIO Verilog test world uses a variant of a top level testbench which is based on a bus compliance testbench from the AMBA Compliance Test Suite. For further information refer to the ARM Micropack AMBA Compliance Test Suite User Guide.

The test bench is designed to be used with most common Verilog simulators, and it is suited to AMBA system designers who want to ensure the portability of their blocks to any other AMBA designs. This simplifies the ASIC integration task and exchange of peripherals between projects.

The test bench is “programmable”, meaning that a description of the transfers to be performed on the **Unit Under Test** (UUT) is given without a need to modify the test bench implementation, hence the “generic” property. This transfer description takes the form of a text file that can be read by the specific test bench to which it is being targeted.

Figure 6.2-1 illustrates the BusTalk Verilog testbench for simulating test vectors.

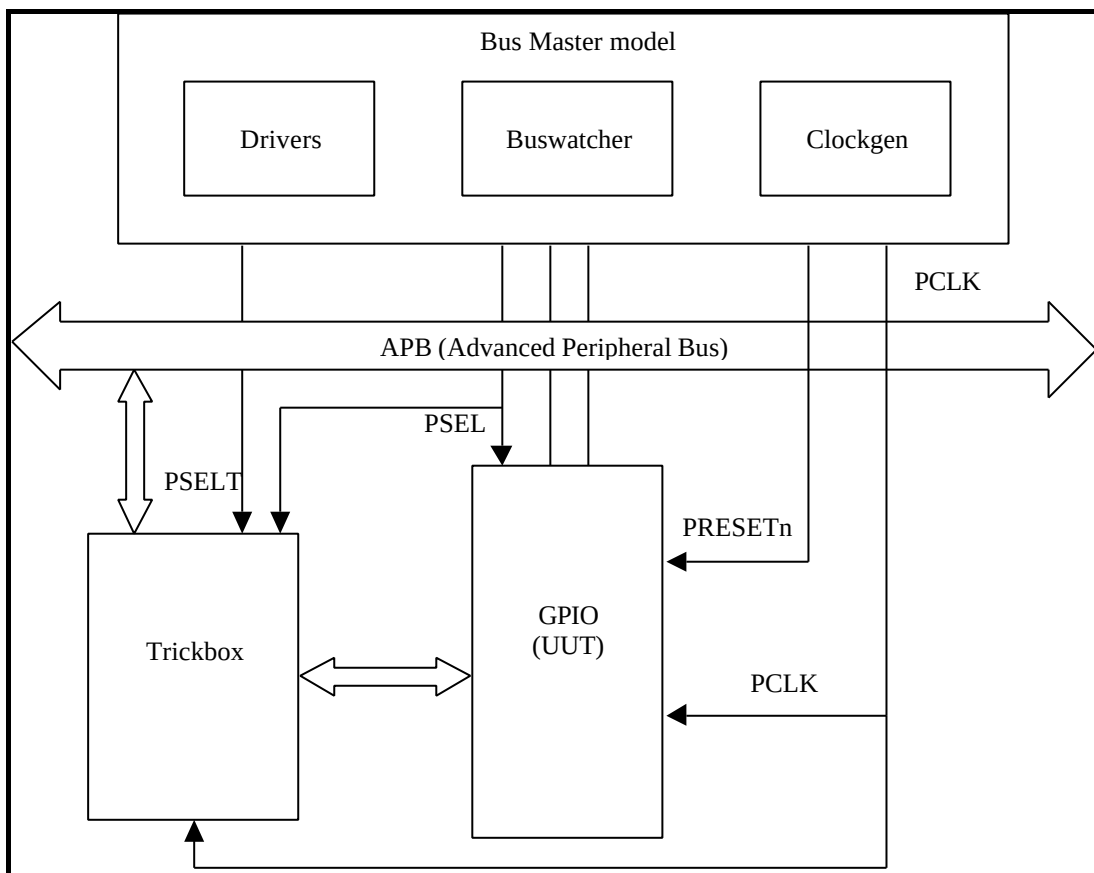


Figure 6.2-1 Verilog Testbench for simulating test vectors

Figure 6.2-2 illustrates the hierarchy for the Verilog testbench.

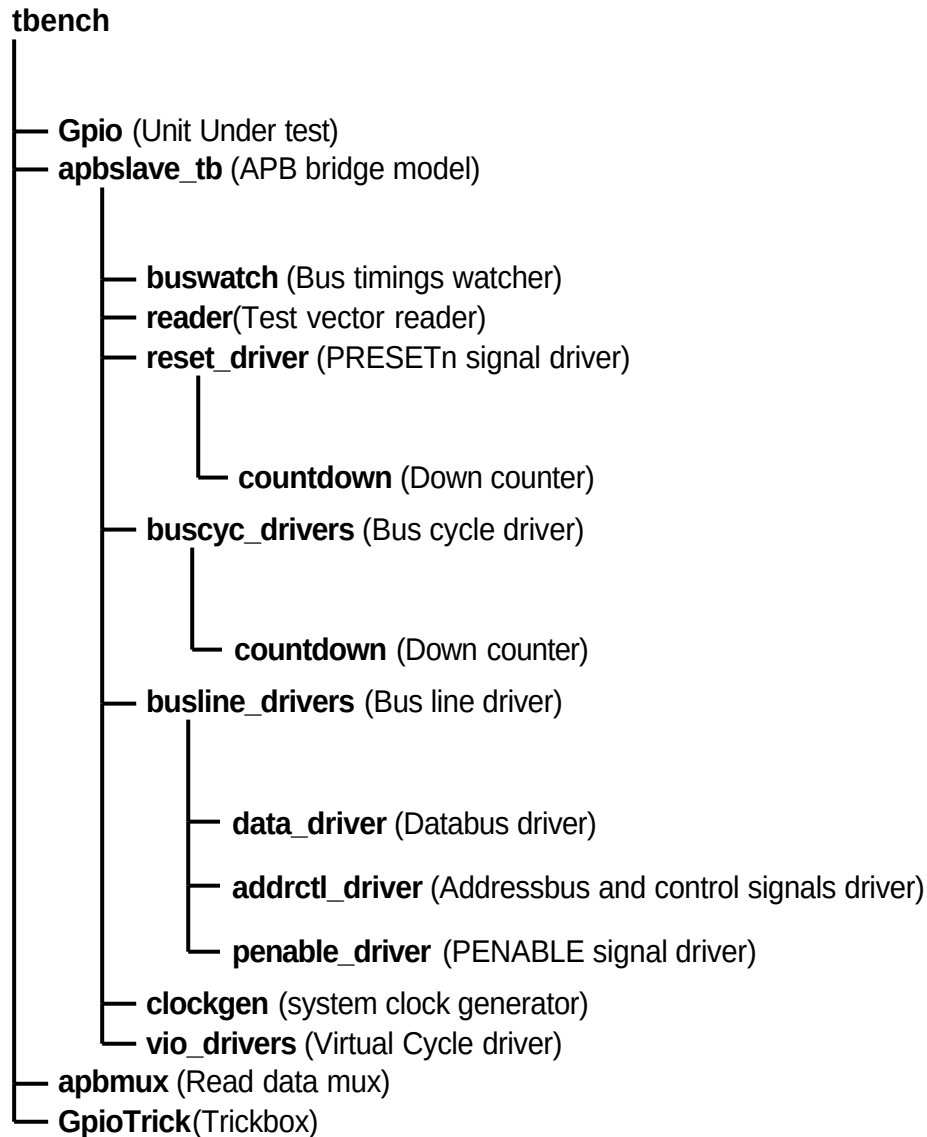


Figure 6.2-2 Verilog Testbench hierarchy

The testbench, **tbench.v** is used for functional simulations of the GPIO. This module instantiates the GPIO (Unit Under Test), the behavioural model of the APB Bridge, the trickbox and the APB Read mux.

The default test frequency is set at 100 MHz. This is the same frequency to which the GPIO was constrained during synthesis. **PCLK** is set to the frequency defined in timing.v using the Tclkh and Tckl 'define values when instantiating apbslave_tb.v in tbench.v. Note that the 'define values override the local parameters defined in clockgen.v in the **verification/verilog/common** directory.

6.2.1 Unit Under Test

The Unit Under Test may be one of the following

- GPIO Verilog RTL
- GPIO Verilog netlist from Verilog Source
- GPIO Verilog netlist from VHDL Source

One out of these three versions may be referenced via the corresponding links to uut directory in the **verification/verilog** directory.

6.2.2 APB Bridge Model

The UUT is stimulated primarily by the APB bridge behavioural model. This block issues commands on the APB bus under program control.

The APB bridge model, `apbslave_tb.v` instantiates the buswatcher, the reader, the reset driver, the bus cycle drivers, the bus line drivers and the clock generator. The reader block reads the test vectors from the `bif.sim` file (from the **verification/bustest/invec** directory) one line at a time and drives the information about the bus cycle in the form of packets. If the information driven by the reader indicates that the current cycle is a reset cycle, the reset driver drives the **PRESETn** signal with the correct timings. The `buscyc_drivers` instantiates an APB cycle driver for each cycle type, i.e PSW, PSR, PI, PO, etc. The `busline_drivers` drives the databus, the address and control signals with the correct timings. The buswatch block checks the Read/Write data bus timings. The clockgen generates the system clock, **PCLK**.

The APB bridge model also provides the Select line for the Trickbox.

Any access to the address range

0x60 00 00 00 to 0x60 00 00 FF

will access the Trickbox via the **PSELT** select line.

Accesses to any address outside the above range will select the UUT via the **PSEL** select line.

6.2.3 Trickbox

The functionality of the GPIO is verified via a Trickbox model. The trickbox is an APB Slave which is designed to supply test stimulus to the non-AMBA input pins of the GPIO and capture the responses from the non-AMBA output pins of the same.

6.2.4 Protocol Checker

The testbench includes a protocol checker which reports any protocol or timing violations during accesses to any APB slave. Timing parameters can be set using the `timings.v` file in the **verification/verilog/tbench** directory. The buswatch block which is in the **verification/verilog/buswatcher** directory checks the Read/Write data bus timings and reports any violations in the timings.

6.2.5 APB Multiplexer

The read data buses from the GPIO and the trickbox are multiplexed together in this module before being connected to the APB Bridge model.

6.2.6 Test Vectors

The test vectors for the verification of the GPIO are coded into separate C files depending on the logical region of the GPIO which these vectors are testing. Make options have been provided to use the entire set of test vectors together or to use them individually.

6.2.7 Parameters

The GPIO testbench provides some configurability options via parameters and defines. All the parameters, except **MESG_ENB** and **SUPPRESSONRESET** are configurable through the **tbench.v** file. **MESG_ENB** and **SUPPRESSONRESET** are configurable through the **buswatch.v** that resides in **verification/verilog/buswatcher**. The define, **DATA_CHECK** is configurable through the **apbmux.v** that resides in **verification/verilog/tbench**.

XonPSEL

XonPSEL is used in the **tbench.v** file that resides in **verification/verilog/tbench**.

This define is used to eliminate the 'X's that appear on the **PSEL**, **PWRITE** and **PADDR** lines when these lines are not being actively driven. If **XonPSEL** is set to **0** in the top level of the testbench, these signals will go to '0' when the corresponding line driver is inactive. If set to **1**, the signals go to 'X' when the line driver is idle.

Verbosity

Verbosity is used in the **data_driver.v**, the **reset_driver.v** and the **buscyc_drivers.v** which reside in **verification/verilog/bid**.

This parameter is used to control the verbosity of the transcript generated during simulation. If **Verbosity** is set to **1**, every command issued will have a corresponding message in the transcript file. Every poll operation in a POLL command sequence is also reported. If **Verbosity** is set to **0**, a message is issued only if an error occurs on a read. Similarly during a POLL command, a message is flagged only if a POLL time-out occurs.

HaltOnMismatch

This is used in the **data_driver.v** which resides in **verification/verilog/bid**.

This parameter is used to stop the simulation if a read is unsuccessful. If set to **1**, the simulation halts after an error is reported. If set to **0**, the error is flagged but the simulation continues.

SUPPRESSONRESET

SUPPRESSONRESET is used in the **buswatch.v** which resides in **verification/verilog/buswatcher**.

This parameter is used to suppress the checking of the **PRDATA** timings when **PRESETn** signal is asserted. If **SUPPRESSONRESET** is **1**, the **PRDATA** set-up and hold timing violations are not flagged when **PRESETn** is asserted. If **SUPPRESSONRESET** is set to **0**, **PRDATA** set-up and hold timing violations are flagged irrespective of **PRESETn**.

MESG_ENB

MESG_ENB is used in the **buswatch.v** which resides in **verification/verilog/buswatcher**.

This parameter is used to enable or disable the flagging of error messages when **PRDATA** does not go to zero when the UUT is not selected. The flagging of these error messages is disabled when this parameter is set to **0**. If **MESG_ENB** is set to **1**, the error messages are flagged whenever **PRDATA** goes to a non-zero value when the UUT is not selected. This generic will have to be set to **0** in testbenches which include more than one slave.

DATA_CHECK

DATA_CHECK is used in the **apbmux.v** which resides in **verification/verilog/tbench**.

This define is used to enable or disable the flagging of error messages when the **PRDATA** driven by a slave has a non-zero value when it is not selected. When **DATA_CHECK** has a value **1**, these error messages are flagged.

When it has a value **0**, the error messages are not flagged. This define is used in testbenches which include more than one slave.

6.2.8 Running Simulations

The user's guide gives step by step instructions for running simulations using the GPIO test vectors on the Verilog source code and the Verilog netlist.

Run time logs for the RTL simulations are:

- **verification/verilog/Gpio_rtl/Gpio_rtl_modelsim.log**
- **verification/verilog/Gpio_rtl/Gpio_rtl_LDV.sim**

Run time logs for the netlist simulations are:

- **verification/verilog/Gpio_noscan_vhdl_net_min/Gpio_noscan_vhdl_net_min_modelsim.log**
- **verification/verilog/Gpio_scanready_verilog_net_typ/Gpio_scanready_verilog_net_typ_LDV.log**

The code coverage results are:

- **verification/verilog/Gpio_rtl/vnavigator.summary**

6.3 Trickbox Description

The trickbox is a programmable APB device designed to provide a test engineer a means to drive the non-APB input pins of the GPIO and also to sample the non APB output responses of the same.

The following sections detail the functionality of the trickbox.

6.3.1 Trickbox Signal Description

Name	Type	Source / Destination	Description
PCLK	Input	From Test Bench	APB clock
PRESETn	Input	From Test Bench	Bus reset signal, Active Low
PA[7:2]	Input	From Test Bench	APB address bus
PRDATA[7:0]	Output	To Test Bench	APB read data bus
PWDATA[7:0]	Input	From Test Bench	APB write data bus
PSELT	Input	From Test Bench	Trickbox select signal from decoder. This select signal is used for Trickbox-specific register access.
PENABLE	Input	From Test Bench	APB enable signal. PENABLE is asserted HIGH for one cycle of PCLK to enable a bus transfer cycle.
PWRITE	Input	From Test Bench	APB transfer direction signal, indicates a write access when HIGH, read access when LOW.

Table 6.3-1 APB signal descriptions

Name	Type	Source/Destination	Description
nGPEN[7:0]	Input	From GPIO	GPIO Output pad enable signal
GPOUT[7:0]	Input	From GPIO	GPIO Output data signal driver
GPIN[7:0]	Output	To GPIO	GPIO Input data from pad
nGPAFEN[7:0]	Output	To GPIO	GPIO Hardware control output enable
GPAFOUT[7:0]	Output	To GPIO	GPIO Hardware control data input
GPAFIN[7:0]	Input	From GPIO	GPIO Hardware control data output
GPIOINTR	Input	From GPIO	GPIO Interrupt signals
GPIOMIS[7:0]	Input	From GPIO	Combined OR version of GPIOMIS

Table 6.3-2 GPIO signals

6.3.2 Trickbox functional description

The functionality of the GPIO is verified via a Trickbox. **Figure 6.3-1** shows the basic blocks of the Trickbox in connection with the GPIO.

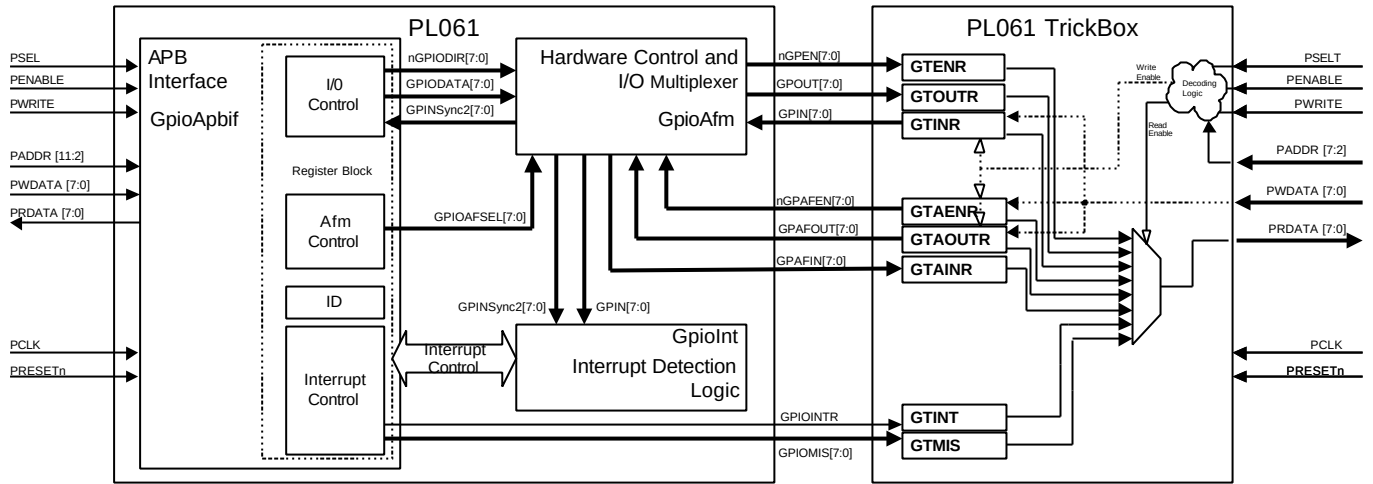


Figure 6.3-1 GPIO Trickbox Block Diagram in connection with the GPIO

Trickbox register summary

Address (Offset from Trickbox Base)	Type	Width (bits)	Reset Value	Name	Description
0x00	R	8	-	GTENR	Status of GPIO Output Enables
0x04	R	8	-	GTOUTR	Status of GPIO Outputs
0x08	R/W	8	0x00	GTINR	Controls GPIO Inputs
0x0C	R/W	8	0x00	GTAENR	Controls GPIO Alt. Functionality O/P Enables
0x10	R/W	8	0x00	GTAOUTR	Controls GPIO Alt. Functionality Outputs
0x14	R	8	-	GTAINR	Status GPIO Hardware Control Inputs
0x18	R	1	-	GTINT	Status of Interrupt Output
0x1C	R	8	-	GTMIS	Status of Masked Interrupt Status Output

Table 6.3-3 Trickbox register Summary

The Trickbox reads the GPIO output pins through the GTOUTR register. The status of the Output Enable pins is read through the GTENR. The input pins of the GPIO, **GPIN[7:0]**, are connected to the read/writable register GTINR of the Trickbox. Hardware Control is simulated through GTAENR, GTAOUTR and GTAINR. The Interrupt pin of the GPIO, **GPIOINTR**, is connected to the read only register GTENR of the Trickbox. Similarly, the GPIO Masked Interrupt Status is connected to GTMIS.

6.4 Functional Tests

The verification code is written in C using BusTalk commands to control the AMBA interface to the peripheral. To exercise non-AMBA interface signals a Trickbox is provided which allows BusTalk commands to control and monitor all the peripheral's I/O signals.

To aid understanding the test program is split into a number of functions, each contained in a separate source file. These functions are then called by the 'main' function, which is contained in the Gpio.c source file.

The following sections detail the verification to be performed on the PrimeCell GPIO PL061. Statement, branch and toggle coverage should be as close to 100 % as possible. Any deviation from this is documented in

verification/vhdl/Gpio_rtl/README.txt

verification/verilog/Gpio_rtl/README.txt

for the vhdl and verilog Gpio source code.

6.4.1 Reset State : RstTest.c

All programmer visible registers within the GPIO model will be checked, ensuring that they reset to the expected state (**0x00**) defined in the Device Specification, [Reference 1]. The Testbench will reset the GPIO by driving **PRESETn** low.

The interrupt detection logic of the device awakens from reset with falling-edge detection as the default. Therefore, the reset value of **GPIORIS** (Raw Interrupt Status) is dependent on the level of the pins. GPIO pins driven low after reset will cause the corresponding pins in **GPIORIS** being '1'.

It must be verified that when the GPIOIE (Interrupt Enable) register is zero, no active bits in **GPIORIS** (Raw Interrupt Status) are seen on **GPIOINTR** and **GPIOMIS**.

Using the Trickbox data patterns will be applied to the GPIO pins to verify that they do not trigger interrupts, nor output is obtained in the Hardware Control lines (non-AMBA). Furthermore, no value written through the non-AMBA lines will appear on the GPIO pins.

Hardware Control after reset will be verified in the following section 6.4.5.

Identification Registers **PERIPHERALID** and **PRIMECELLID** are hard coded and the fields within the registers will determine the reset value.

6.4.2 Register Access: TestGPIOReg.c

Checks will be performed on all programmer visible registers within the GPIO model, ensuring that registers are readable/writable according to the Device Specification, see [Reference 1, 3.2 Summary of PrimeCell GPIO registers].

Data patterns will be written to the writable registers such that all the bits of the register are toggled from 0 to 1 and 1 to 0 at least once.

- Read only and write only registers require special consideration when verifying their features.
- Write only registers will be tested by means of its functionality being observed through the Trickbox.
- Similarly, read only registers will be checked after executing strategic events that should cause changes on them. Accessing them directly via the address map and performing write operations will make sure that no change in value occurs.

Data patterns are written to each of the read/writable registers GPIODIR, GPIODATA, GPIOIS, GPIOIBE, GPIOIEV, GPIOIE and GPIOAFSEL. The register contents are read back to see if it contains the written value. The data patterns are written such that all the bits of the registers are toggled from 0 to 1 and from 1 to 0 at least once.

GPIOINTC is a write-only register, its function is to clear the Edge-Detected loop. GPIORIS and GPIOMIS, both of them read only registers, return the status of the interrupt detection logic.

GPIOINTC together with GPIORIS and GPIOMIS are a group of dedicated registers dependent on the interrupt functionality of the GPIO. Therefore, the accessibility of these registers can only be verified combined with the functional effects of their status, see section 6.4.6 below.

GPOTCR is a one-bit R/W register, GPOTOP1 is an 8-bit write-only register, that affects the value of GPIOMIS when ITEN(GPOTCR[0]) is 'high'. Finally, GPOTOP2 is a read-only register reflecting the status of GPOTNTR interrupt output. GPOTOP2 is accessible independently of GPOTCR. See **Figure 6.4-1**.

Identification Registers PERIPHERALID and PRIMECELLID are both 32-bit wide hard coded registers. They have been implemented as 4x8-bit read only registers, such that they are addressed and read in four consecutive address locations.

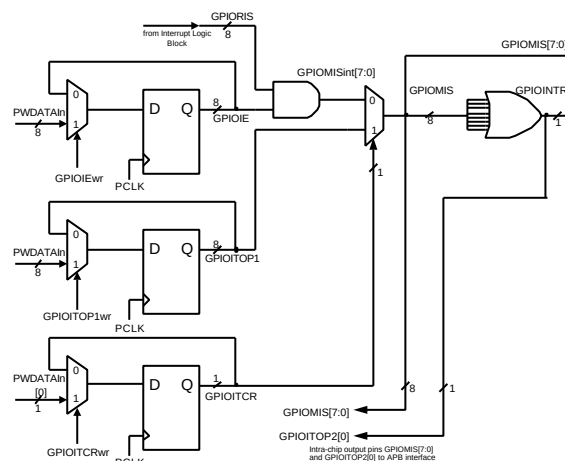


Figure 6.4-1: Implementation of Integration Test Vector Registers

6.4.3 Register Access (GPIODATA Address Masking) : TestGPIODATAREgMasking.c

Special attention will be given to the address masking of the Data register using offsets 0x000 to 0x3FC (256 words). During a write, if the address bit associated with that data bit is high, the value of the GPIO register can be altered; if it is 0, it will be left unchanged. Similarly, during a read if an address bit associated with data is high the value will be read; if it is 0(low) it will be read as zero. This is illustrated in [Reference 1, Figures 2-2 and 2-3].

GPIODATA, the Data register of the GPIO, is addressable in 256 memory locations using offsets 0x000 to 0x3FC (256 words). Every memory location offers a characteristic mask formed from **PADDR[9:2]** address lines.

This test first verifies that **GPIODATA** is read/writable using the mask **0xFF**(8-bit data is wholly written to **GPIODATA** and the content of **GPIODATA** can be read entirely).

The test then concentrates in the write mask. Writing **0xFF** to different memory locations and reading the content of **GPIODATA** through **+0x3FC** it can be verified that the value written in **GPIODATA** was **PADDR[9:2]** in each case. Notice that the contents of **GPIODATA** must be cleared before each new write operation in this test.

A zero in the mask avoids the write operation to occur in the corresponding **GPIODATA** bit. This is verified writing **0x00** to **+0x000** and then reading back the previous value, which in the test was **0xFF**.

Read mask functionality is exercised and verified via reading a known data value, **0xFF**, from **GPIODATA** at different memory locations and verifying that it equals to the corresponding **PADDR[9:2]**.

6.4.4 Interrupt Generation Capability : IntTests.c

Three dedicated registers allow selecting the source of the interrupt, its polarity and edge properties. When one or more GPIO lines causes an interrupt, interrupt outputs GPIOINTR and the corresponding GPIOMIS are activated. Software can detect the source of the interrupt in GPIOMIS and then clear the interrupt (edge case only) in order to enable the circuit for further interrupts. For a level case, it is assumed that the external source will hold the level constant for the interrupt to be recognised by the processor.

Table 6.4-1 GPIO Interrupt registers

0x404	GPIOIS (Interrupt Sense Reg.)	GPIO Interrupt Edge/Level 0 = Edge 1 = Level	R/W
0x408	GPIOIBE (Interrupt Both Edges Reg.)	GPIO Interrupt Both Edges Only applies when Edge =1, value of GPIOEV is ignored when this bit is set.	R/W
0x40C	GPIOIEV (Interrupt Event Reg.)	GPIO Interrupt Event 1 = Rising Edge/ HIGH Level 0 = Falling Edge/ LOW level	R/W
0x410	GPIOIE (Interrupt Enable Reg.)	GPIO Interrupt Enable	R/W
0x414	GPIORIS	GPIO Raw Interrupt Status	RO
0x418	GPIOMIS	GPIO Masked Interrupt Status	RO
0x420	GPIOIC	GPIO Interrupt Clear	WO

Bits set in GPIORIS will be the immediate confirmation that conditions established in GPIOIS, GPIOIBE and GPIOIEV have been met. This is not enough to trigger an interrupt. Consent must be granted by GPIOIE (Interrupt mask) before any bit can be set on GPIOMIS (masked interrupt status) and thus triggering the activation of GPIOINTR.

Two different combinations of working configurations will be able to run concurrently.

- **Line set as output, Software Interrupt triggering.**

An interrupt may be triggered by, setting the suitable conditions with GPIOIS, GPIOBE and GPIOIEV, setting this line to act as output and then driving the output line effectively to match these requirements and thus triggering an interrupt through GPIOINTR. In this case data is driven directly from the APB interface, through GPIODATA, into the Interrupt detection Block GpioInt.

- **Potentially Asynchronous, to PCLK, source of interrupts.**

The following three cases are simulated through GPIN[7:0], simulating the physical implementation functionality of the PAD, as seen in **Figure 6.4-2**.

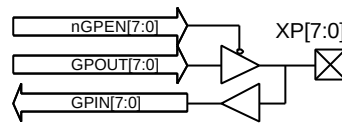


Figure 6.4-2 Implementation of XP[7:0]

- **Line in APB Interface functionality set as input**

Data entering the GPIO is potentially asynchronous to PCLK. Therefore, to avoid metastability problems a synchronisation stage is introduced in the GpioAfm block, see section 5.6.

- **Line in Hardware Control functionality set as input**

The capabilities of the PrimeCell GPIO have been extended to cover interrupt generation on lines acting in Hardware Control. The working characteristics of this configuration match those of the APB Interface functionality.

- **Line in Hardware Control functionality set as output**

Values output through these lines will be able (if not masked) to trigger a Hardware Interrupt if conditions are met as it is explained in section 5.5 .

The interrupt detection capabilities of the GPIO allow the selective detection of the following events on the GPIO pins **GPIN[7:0]**:

LEVEL:

- High Level : The interrupt is active as long as the GPIO pin is being held low.
- Low Level : The interrupt is active as long as the GPIO pin is being held high.

EDGE:

- Rising Edge : The interrupt is active when the GPIO pin is driven from low to high. The interrupt will remain active while the Edge-Detected loop is not cleared via writing a '1' to the corresponding bit in GPIOIC.
- Falling Edge : The interrupt is active when the GPIO pin is driven from high to low. The interrupt will remain active while the Edge-Detected loop is not cleared via writing a '1' to the corresponding bit in GPIOIC.
- Both Edges : The interrupt is active when the GPIO pin is either driven from high to low or driven from low to high. The interrupt will remain active while the Edge-Detected loop is not cleared via writing a '1' to the corresponding bit in GPIOIC.

Each of these cases have been verified isolated, taking into account that the Raw Interrupt Status register should not react to any other event but the one being configured in the Interrupt Control Registers.

The accessibility of GPIOINTC, GPIORIS and GPIOMIS have been inherently tested during these tests.

6.4.5 Hardware Control Functionality : AltFuncTests.c

The verification code will ensure that from the point of view of the APB interface, Hardware Control pins will not cause abnormal operation, and vice versa. The APB interface will have access to the data driven from both Hardware Control lines and external devices connected to the pads. However the contents of the data lines dedicated to the APB interface (GPIOAFSEL(i)=0) will be transparent to Hardware Control lines. Control and data transfer over Hardware Control lines should be made available to the TrickBox by means of **nGPAFEN[7:0]**(Hardware Control Enable lines), **GPAFIN[7:0]**(Hardware Control i/p lines), **GPAFOUT[7:0]**(Hardware Control o/p lines), see **Figure 6.4-3**.

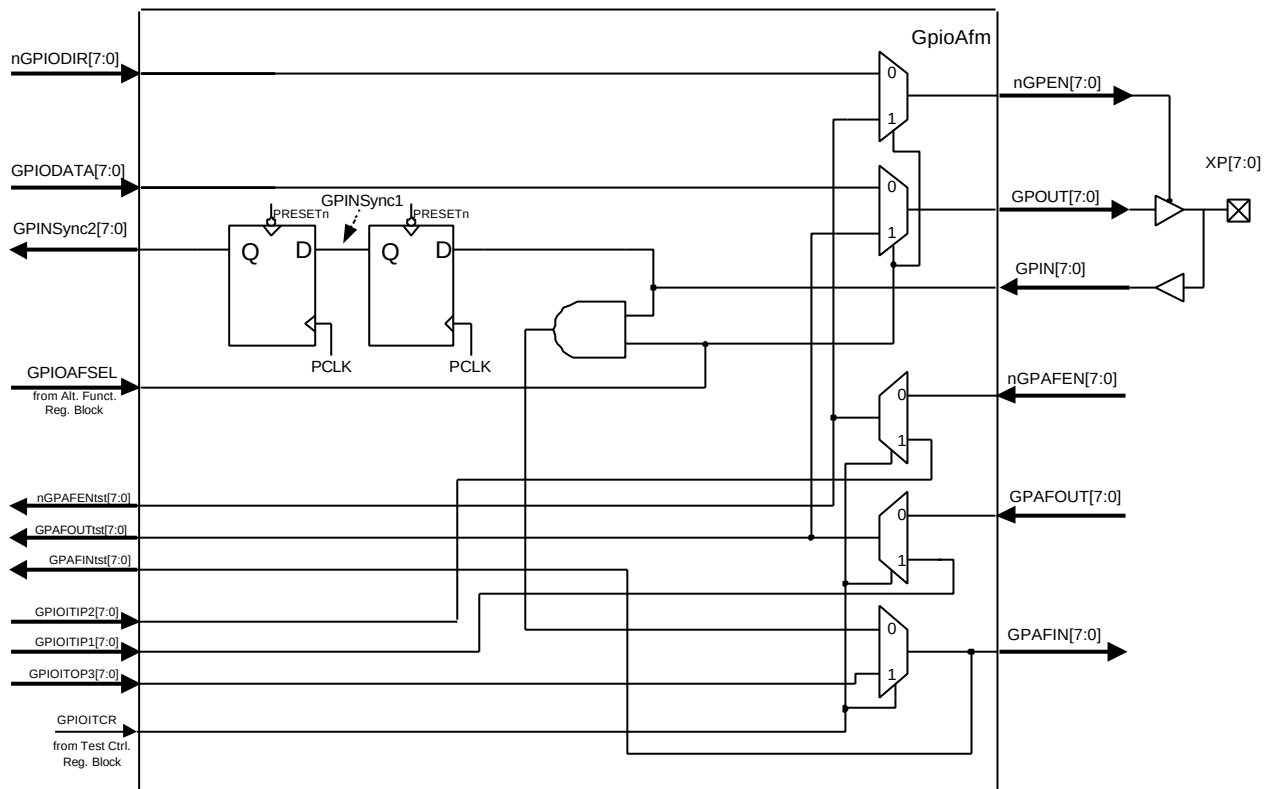


Figure 6.4-3: Hardware Control-APB Interface I/O Multiplexer

The verification procedure for this block is divided into two functional verification tests:

1. All GPIO pins have the same flexibility to act as input or output, as well as, triggering interrupts. **Figure 6.4.5-1** shows how **GPIN[7:0]** before being divided, offers the state of the external pins **XP[7:0]** to the Interrupt Detection Logic. Hence, Interrupt capabilities cover both APB Interface and Hardware Control lines.
2. The AND-gate, which stops APB data flowing in the Hardware Control interfaces avoids line toggling and drives zero instead. This isolating/separating data-flows feature requires also verification.

Multiplexors allocated in Hardware Control lines are verified as part of a dedicated set of tests for Integration Vectors capabilities.

AltFuncTests() test function have been divided in four sections, each one verifying the different Hardware Control configurations:

Line in APB functionality set as output, Software Interrupt triggering

1. Data will flow from the APB interface to the GPIO pins and data in the Hardware Control lines do not interfere.

Potential Asynchronous source of interrupts

The following three cases are simulated through GPIN[7:0], simulating the physical implementation functionality of the PAD, as seen in **Figure 6.4-2**.

2. After Reset, Data will flow to the APB interface (**GPIODATA**) from the GPIO pins and data in the Hardware Control lines do not interfere.
3. Line in Hardware Control set as input. Data will flow from the Alt. Functionality lines to the GPIO pins and data in the APB interface lines are ignored.
4. Line in Hardware Control set as output. Data will flow to the Alt. Functionality lines from the GPIO pins and data in the APB interface lines are ignored.

Interrupt Capabilities cover both APB Interface and Hardware Control lines. The input to this functionality is the value of the GPIO physical pins **XP[7:0]**. Therefore, as all the possible paths to the GPIO pins have been covered, including Hardware Control lines. See **Figure 6.4-2**.

GPIO modes 3 and 4 configure the GPIO to monitor the data flowing in lines set to work in Hardware Control with a resolution of 2*PCLK seconds. However, Hardware Control lines cannot monitor lines set to work in APB mode (GPIOAFSEL=0).

6.4.6 Integration Vectors Functionality: IntVectors.c

Besides the Integration Test Vectors introduced in the Interrupt Control Logic circuitry, a set of registers are required to prove the Hardware Control Intra-chip pins

- **nGPAFEN[7:0]** will be controlled in Integration Test Mode only by GPIOITIP2[7:0]
- **GPAFOUT[7:0]** will be controlled in Integration Test Mode only by GPIOITIP1[7:0]
- **GPAFIN[7:0]** will be controlled in Integration Test Mode only by GPIOITOP3[7:0]

Their implementation is a direct replication of the standard proposition in PrimeCell Integration Vector Strategy[Reference 7], see **Figure 6.4-5**. In order to offer a sense of security the functionality of these registers must be verified.

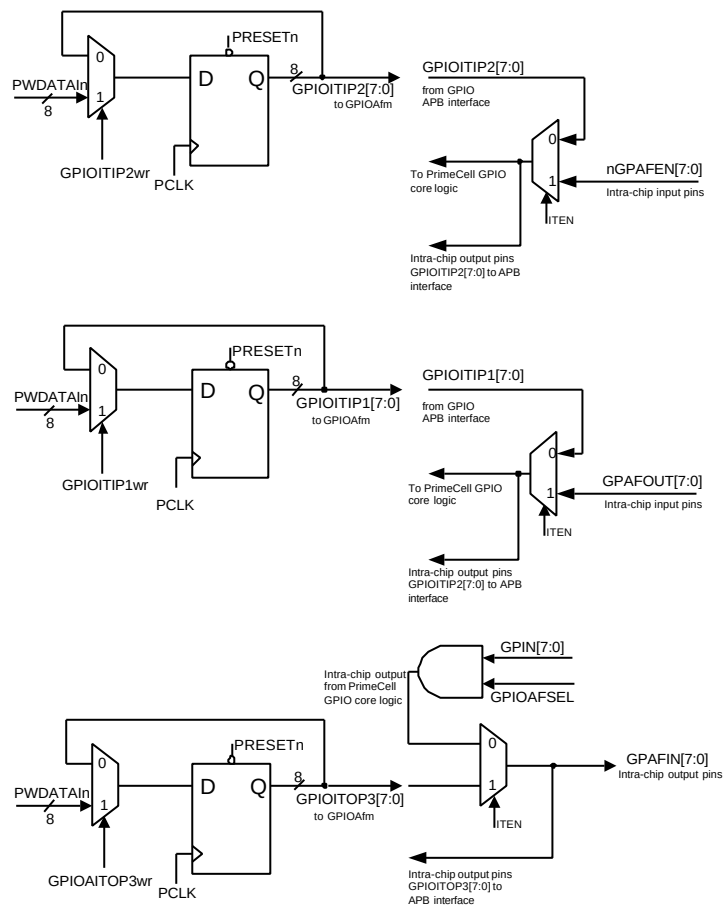


Figure 6.4-5: Hardware Control Integration Test Vector Registers

Primary Inputs and Outputs will be prepared for Integration Tests via the Alternate Capabilities of their Integrated Test Vectors Registers GPIOITIP1 and GPIOITIP2 for **GPOUT[7:0]** and **nGPEN[7:0]** respectively. The Primary Input pins **GPIN[7:0]** will be checked using GPIODATA in Input mode (GPIODIR=0x00).

Note: Data Entering **GPIN[7:0]** is considered potentially asynchronous to PCLK. Therefore, before **GPIN[7:0]** status can be read from GPIODATA a time period of $2 \times \text{PCLK}$ must be allowed.

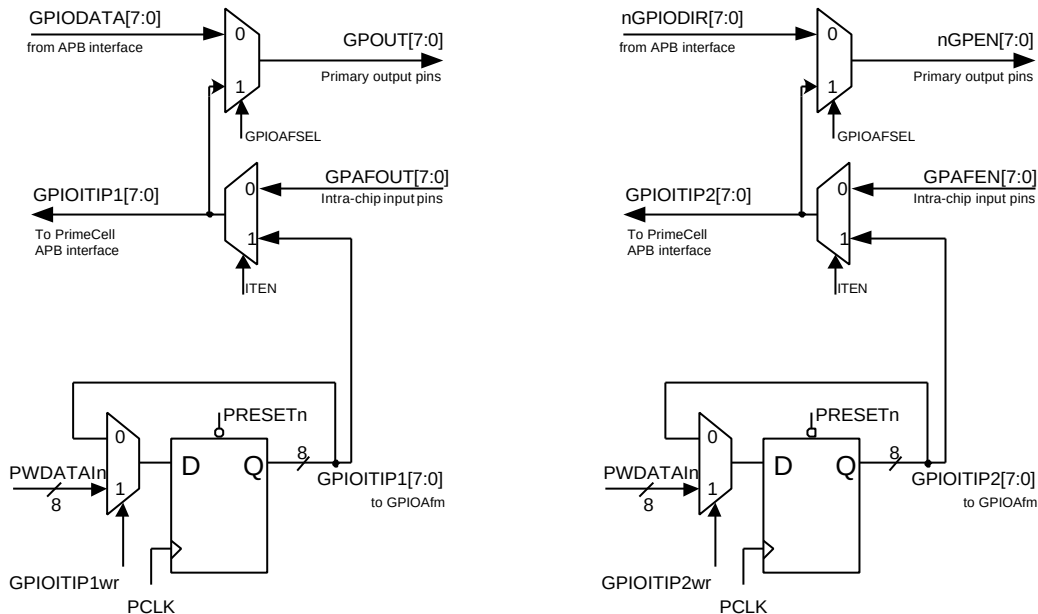


Figure 6.4-6: Integration Test Vectors for Primary Outputs in the PrimeCell GPIO

In order to verify the functionality of the Integration Test Registers GPIOITIP1 and GPIOITIP2 for testing the PrimeCell GPIO primary outputs **GPOUT[7:0]** and **nGPEN[7:0]** the following test will be implemented, see **Figure 6.4-6**.

- Verifying connection between GPIOITIP1 and **GPOUT[7:0]**.

A set of Data patterns will be written to GPIOITIP1. It will be expected to read the same data patterns from the register in the Trickbox used to read **GPOUT[7:0]**, GTOUTR.

- Verifying connection between GPIOITIP2 and **nGPEN[7:0]**.

A set of Data patterns will be written to GPIOITIP2. It will be expected to read the same data patterns from the register in the Trickbox used to read **nGPEN[7:0]**, GTENR.

6.5 Verification Test Vector Generation

A **make sourcefilename** (without the .c extension) in the **verification/bustests** directory will create .bif formatted and .sim formatted vectors in the **verification/bustest/invec** directory. This is the default setting. The makefile in **verification/bustest/invec** directory can be referred for the make options.

make all can be used to create a test vector file including all the tests.

7 GPIO INTEGRATION TEST DETAILS

The GPIO integration test vector suite allows the integrator to verify that the GPIO has been connected into the target system correctly. These vectors test 3 classes of signals:

- AMBA signals (GPIO signals connected directly to the APB)
- Intra-Chip signals (GPIO signals connected to the Interrupt Controller and to the generic peripheral/device connected to the hardware control lines)
- Primary I/O signals (GPIO signals connected to the chip pads)

For more details on Integration testing, please refer to the PrimeCell Integration Vector Strategy document [Ref. 7].

The GPIO Integration testbench is based on an AHB TICTalk testbench. The VHDL and Verilog TICTalk testbenches are used to verify that integration vectors correctly execute in a typical ASIC environment. The VHDL and Verilog TICTalk testbench files reside in the *integration/vhdl* and *integration/verilog* directories respectively. The test vectors that are applied to the TICTalk testbenches are in the *integration/bustest* directory. The testvectors for integration testing are written in BusTalk and then converted into *.tif* format to be applied to the TICTalk integration testbench.

7.1 GPIO VHDL Integration testbench

The AMBA TICTalk VHDL testbench used for integration testing of the GPIO is located in the *integration/vhdl/tbench* directory. The Integration test vectors are located in the *integration/bustest* directory.

The test vectors used with this testbench can be applied to the GPIO when it is part of an AMBA system design.

Figure 7.1-1 illustrates the testbench to apply TICTalk test vectors.

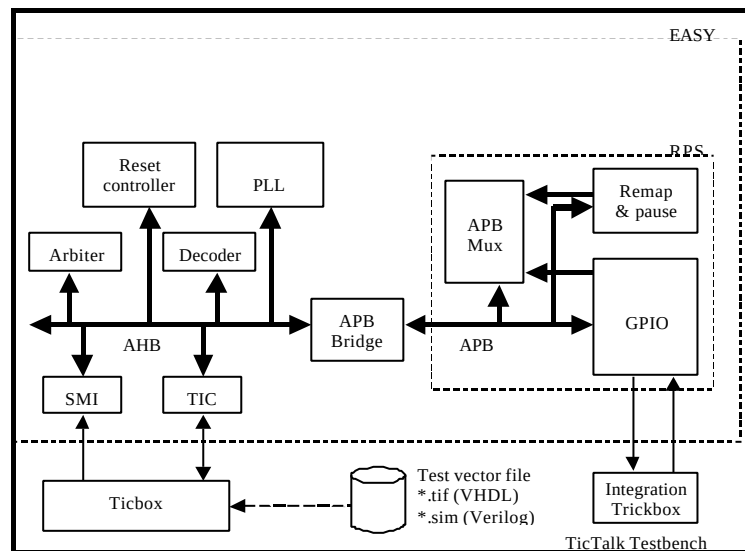


Figure 7.1-1 Testbench for simulating TICTalk test vectors

The GPIO VHDL Integration testbench is a stripped down version of the standard EASY system testbench. For further information refer to the Example AMBA System user guide which is part of ARM Micropack documentation [Reference 5].

Figure 7.1-2 illustrates the hierarchy for the TICTalk testbench.

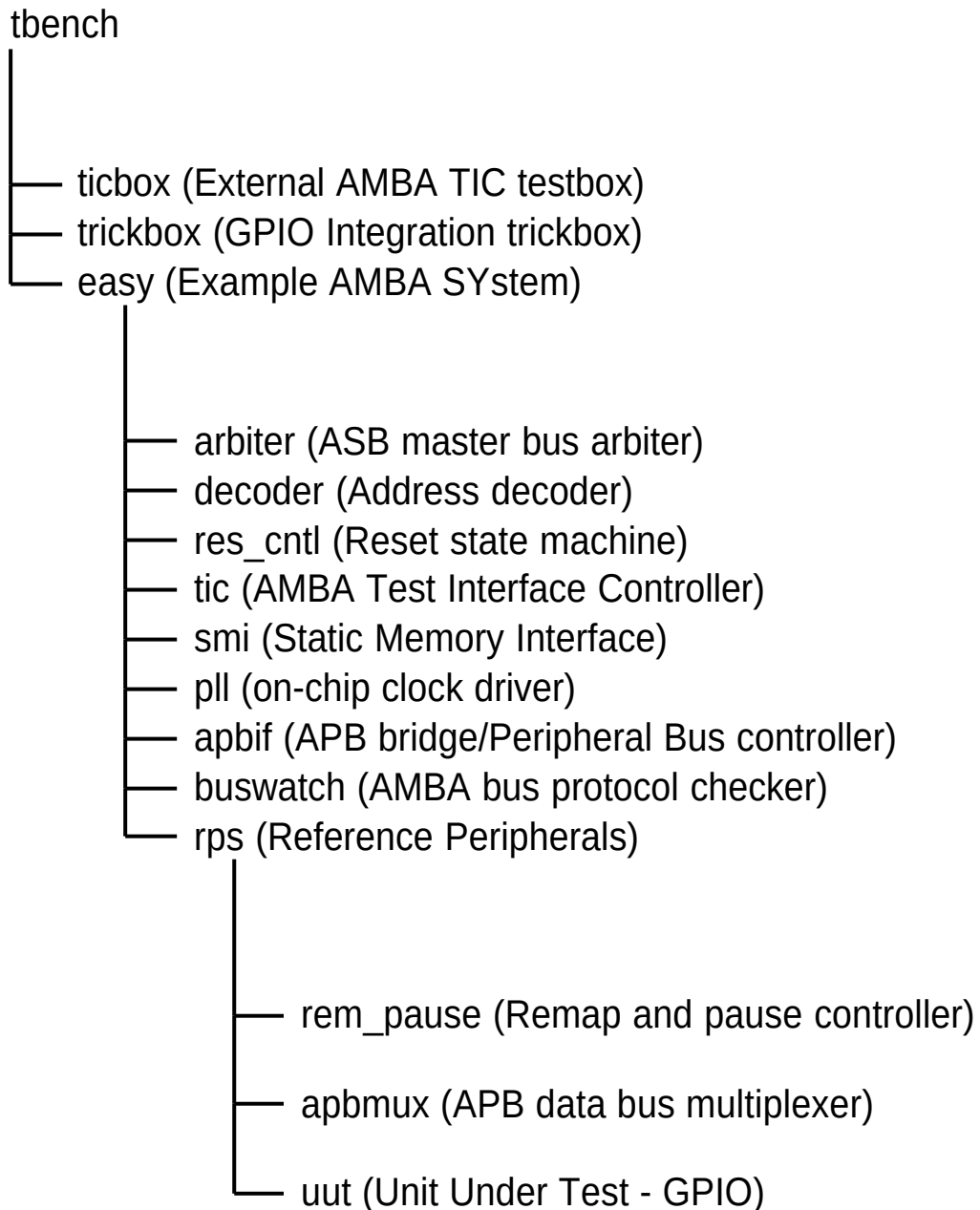


Figure 7.1-2 Testbench hierarchy for simulating TICTalk test vectors

The test vectors are applied to **tbench.vhd**, the top-level of the testbench which is used for running the integration tests of the GPIO. This module instantiates the model of the EASY microcontroller, the integration trickbox and the ticbox. The GPIO is instantiated in the **rps.vhd**, which is a part of the EASY system.

The integration trickbox is used to give a loopback facility to test I/O pins. For further details on the integration trickbox, refer to **Section 7.3**.

7.1.1 Unit Under Test

The Unit Under Test may be one of the following

- GPIO VHDL RTL
- GPIO VHDL netlist from VHDL RTL Source

One out of these two versions may be referenced via the corresponding link to uut directory in *integration/vhdl*.

7.1.2 Test Vectors

The test vectors for the integration testing of the GPIO are coded into separate C files for testing the reset values of all the GPIO registers and for Integration testing. Make options are provided to run the tests together or individually.

7.1.3 Running Simulations

The **README** file in the *integration/vhdl/tbench* directory gives step by step instructions for running simulations using the GPIO Integration test vectors on the VHDL source code.

Run time logs for all the combinations are available in the *integration/vhdl/Gpio_rtl* and *integration/vhdl/Gpio_scaninsert_vhdl_net_max* directories.

Gpio_rtl_modelsim.log

Gpio_scaninsert_vhdl_vhdl_net_max_modelsim.log

7.2 GPIO Verilog Integration testbench

The AMBA TICTalk Verilog testbench used for integration testing of the GPIO is located in the ***integration/verilog/tbench*** directory. The Integration test vectors are located in the ***integration/bustest/invec*** directory.

The test vectors used with this testbench can be applied to the GPIO when it is part of an AMBA system design.

Figure 7.2-1 illustrates the testbench to apply TICTalk test vectors.

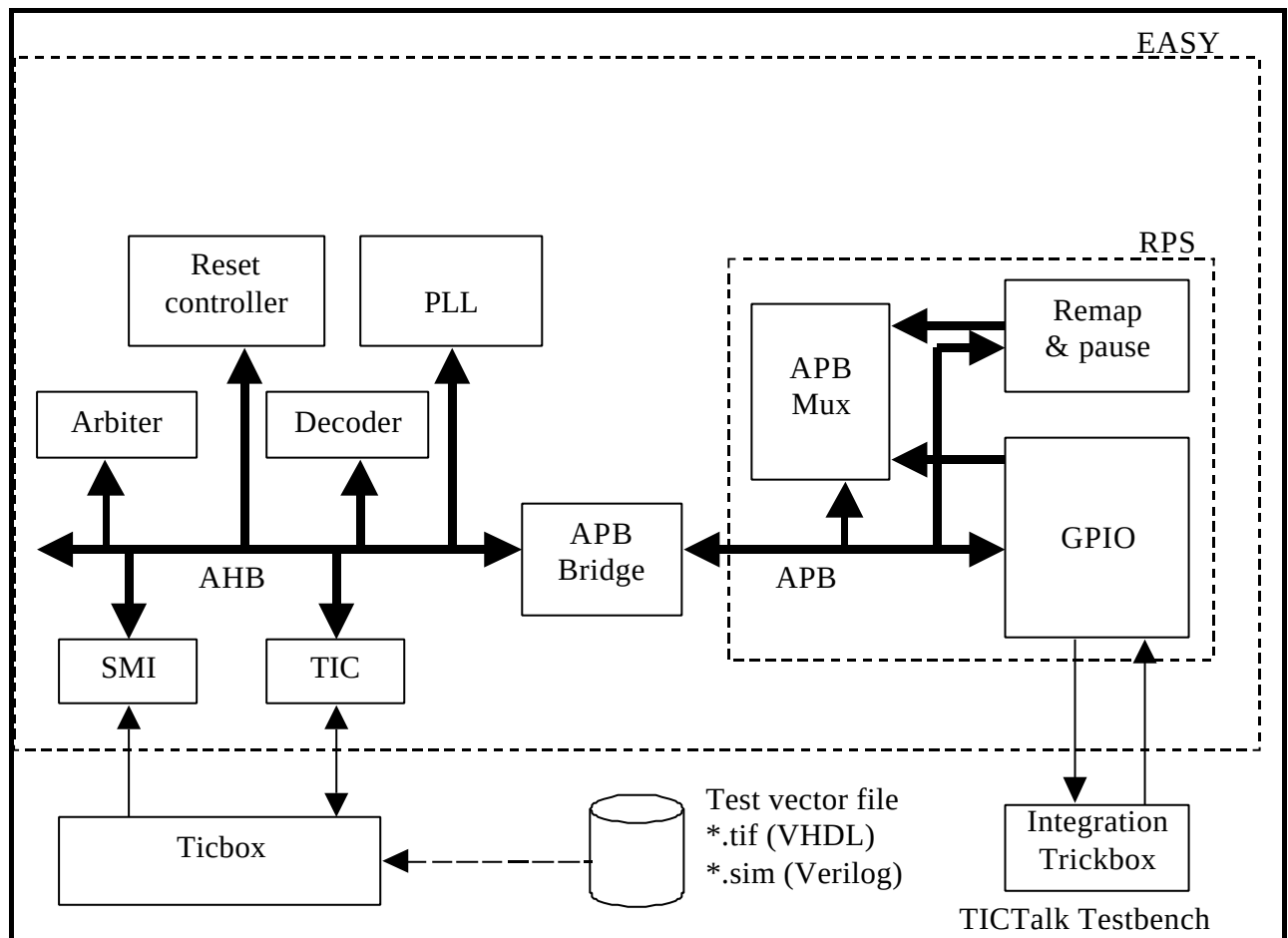


Figure 7.2-1 Testbench for simulating TICTalk test vectors

The GPIO Verilog Integration testbench is a stripped down version of the standard EASY system testbench. For further information refer to the Example AMBA System user guide which is part of ARM Micropack documentation [Reference 5].

Figure 7.2-2 illustrates the hierarchy for the TICTalk testbench.

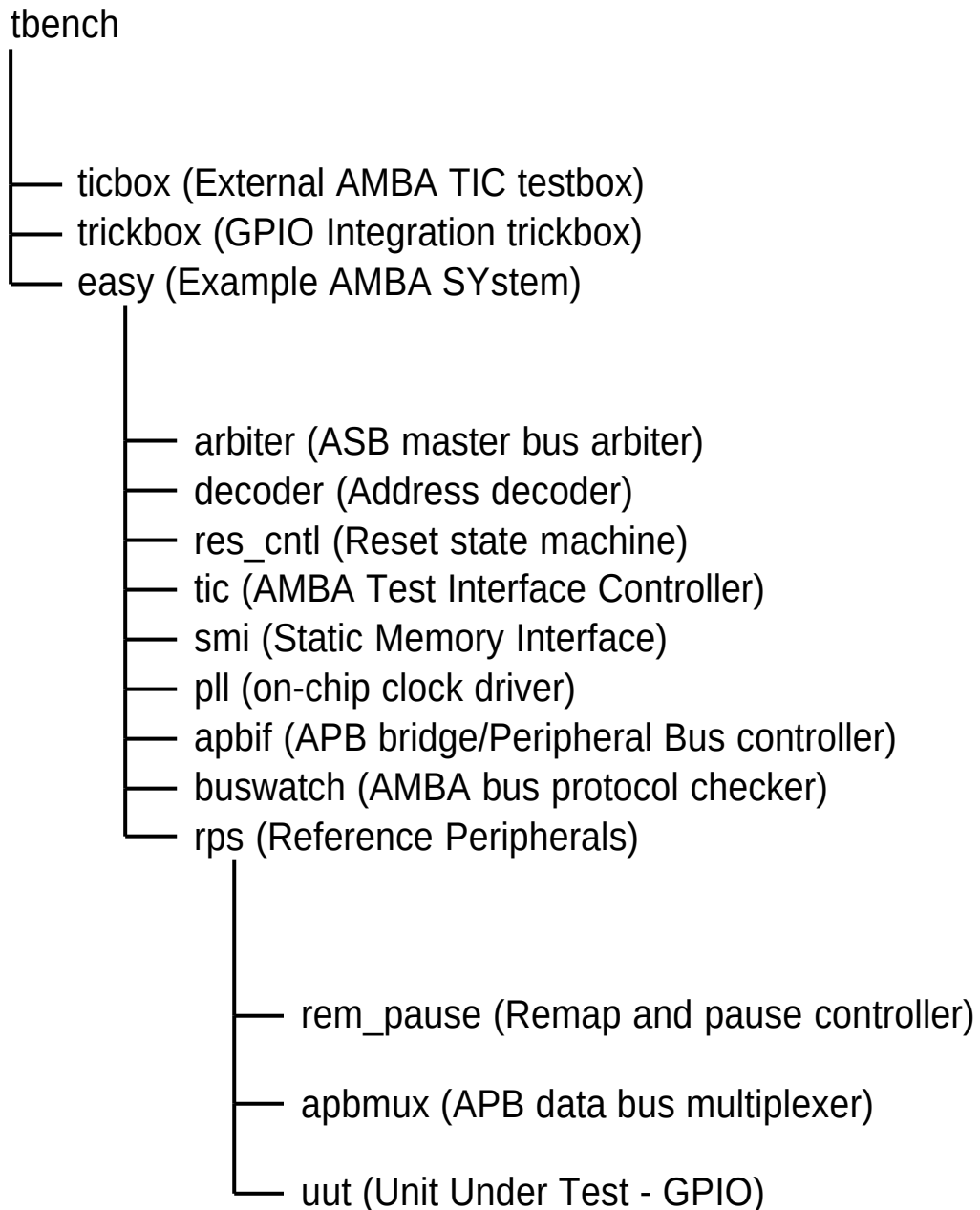


Figure 7.2-2 Testbench hierarchy for simulating TICTalk test vectors

The test vectors are applied to **tbench.v**, the top-level of the testbench which is used for running the integration tests of the GPIO. This module instantiates the model of the EASY microcontroller, the integration trickbox and the ticbox. The GPIO is instantiated in the **rps.v**, which is a part of the EASY system.

The integration trickbox is used to give a loopback facility to test I/O pins. For further details on the integration trickbox, refer to **Section 7.3**.

7.2.1 Unit Under Test

The Unit Under Test may be one of the following

- GPIO VERILOG RTL
- GPIO VERILOG netlist from VHDL RTL Source
- GPIO VERILOG netlist from VERILOG RTL Source

One out of these two versions may be referenced via the corresponding link to the uut directory in *integration/verilog* .

7.2.2 Test Vectors

The test vectors for the integration testing of the GPIO are coded into a separate C files for testing the reset values of all the GPIO registers and for Integration testing. Make options are provided to run the tests together or individually.

7.2.3 Running Simulations

The **README** file in the *integration/verilog/tbench* directory gives step by step instructions for running simulations using the GPIO Integration test vectors on the VERILOG source code.

Run time logs for all the combinations are available in the following files in the *integration/verilog/Gpio_rtl*, *integration/verilog/Gpio_noscan_vhdl_net_min* and *integration/verilog/Gpio_scanready_verilog_net_typ* directories.

Gpio_rtl_modelsim.log

Gpio_rtl_LDV.log

Gpio_noscan_vhdl_verilog_net_min_modelsim.log

Gpio_scanready_verilog_verilog_net_typ_LDV.log

7.3 Integration Trickbox

Integration testing of primary outputs and primary inputs are carried out with the help of the Integration Vector Trickbox.

7.3.1 Integration Trickbox Signal Description

Name	Type	Source / Destination	Description
GTENR	Input	from GPIO	Status of GPIO Output Enables
GTOUTR	Input	from GPIO	Status of GPIO Outputs
GTINR	Output	to GPIO	Controls GPIO Inputs

Table 7.3-1 GPIO Integration Trickbox Signal Description

7.3.2 Integration Trickbox Functional Description

NGPEN, GPOUT and GPIN pin connections are verified as follows. Primary output pins nGPEN[7:0] and GPOUT[7:0] are XOR-ed together and routed back to the primary input pin GPIN[7:0] via the Integration Vector Trickbox. **Figure 7.3-1** shows the block diagram of the GPIO Integration Vector Trickbox.

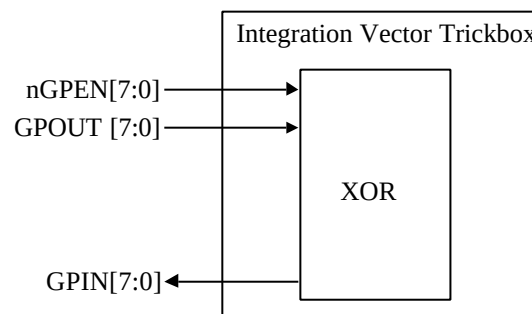


Figure 7.3-1 GPIO Integration Vector Trickbox

7.4 Integration Tests

When the GPIO is used in the AMBA system, it must be ensured that all the I/O pins are correctly connected. Integration vectors allow the user to verify that the GPIO has been wired into the system correctly.

This test is performed with Gpio_Integ() and RstTest() function calls.

The RstTest() function call is used to increase the toggle coverage of the address lines. To achieve this, the reset input (PRESETn) are asserted. When the reset inputs are deasserted all the registers in the GPIO are read back to check for the default values.

7.4.1 Pad signals nGPEN[7:0], GPOUT[7:0] and GPIN[7:0]

Testing of Pad connections (nGPEN[7:0], GPOUT[7:0] and GPIN[7:0]) :

The procedure is given below.

1. Write 0xFF to the Hardware Control register GPIOAFSEL and 1 to the ITEN bit in the Integration Control register. This selects the test path from the GPIOITIP2[7:0], GPIOITIP1[7:0] to the internal nGPEN[7:0], GPOUT[7:0] PAD signals respectively.
2. Make sure the GPIO is in input mode by writing 0x00 to GPIODIR, otherwise the data path from GPIN[7:0] to GPIODATA would be broken and no data could be read.
3. Write a 1 and then a 0 to the GPIOITIP2[7:0] and GPIOITIP1[7:0].
4. The TrickBox will generate the input signals for the GPIN(7:0) pins of the GPIO as a XOR logical operation of the GPIO output lines nGPEN[7:0] and GPOUT[7:0]. See **Table 7.4-1**

nGPEN[i]	GPOUT[i]	GPIN[i]
0	0	0
0	1	1
1	0	1
1	1	0

Table 7.4-1 $GPIN(i) = nGPEN[i] \text{ XOR } GPOUT[i]$

5. Allow a time period of twice PCLK for the data to go through the synchronisation stage in the GpioAfm block.
6. Read GPIODATA+0x3FC and verify that the result is the expected.

7.4.2 Integration Testing of intra-chip signals nGPAFEN[7:0], GPAFOUT[7:0] and GPAFIN[7:0]

When Integration tests are run with the GPIO in a stand-alone test set-up, the procedure for testing of Intra-Chip pins nGPAFEN[7:0], GPAFOUT[7:0] and GPAFIN[7:0] is given below:

1. Write a 1 to the ITEN bit in the Integration Control register. This selects the test path from the GPIOITIP2[7:0], GPIOITIP1[7:0] and GPIOITOP3[7:0] register bits to the internal nGPAFEN[7:0], GPAFOUT[7:0] and GPAFIN[7:0] signals respectively.
2. Write a 1 and then a 0 to the GPIOITIP2[7:0], GPIOITIP1[7:0] and GPIOITOP3[7:0] register bits and read the same register bit to ensure that the value written is read out.
3. When integration tests are run in an integrated system '1' and '0' are actually written into internal registers of the device connected to the Hardware Control lines so as to toggle the nGPAFEN, GPAFOUT from the Generic peripheral.

Note: When the tests are run in an integrated system, the user is expected to replace these write/read operations, with a suitable command to set/read the peripheral Hardware Control signal/s from the peripheral/s connected to these Hardware Control line/s.

4. GPAFIN signal will be controlled from the Gpio via writing to GPIOITOP3 and the value written must be read from the peripheral connected to the Hardware Control lines to verify connectivity.

7.4.3 Intra-chip outputs GPIOINTR and GPIOMIS[7:0]

When Integration tests are run with the GPIO in a stand-alone test set-up, the procedure for testing of Intra-Chip pins GPIOINTR and GPIOMIS[7:0] is given below.

1. Write 1 to the ITEN bit in the Integration Control register. This selects the test path from the GPIOITOP1[7:0] to the internal GPIOMIS[7:0], the value driven externally to the GPIOMIS[7:0] can be read via reading to the GPIOMIS register.
2. GPIOINTR is a combined output resulting from the OR-logical operator on the particular bits of GPIOMIS:

$$\text{GPIOINTR} = \text{GPIOMIS}[7] + \text{GPIOMIS}[6] + \text{GPIOMIS}[5] + \text{GPIOMIS}[4] + \text{GPIOMIS}[3] + \text{GPIOMIS}[2] + \text{GPIOMIS}[1] + \text{GPIOMIS}[0]$$
3. Write a 1 and then a 0 to the GPIOITOP1[7:0] and read from the GPIOMIS register. This will toggle both GPIOMIS[i] and GPIOINTR lines between the Gpio and the Interrupt Controller.
4. The status value of GPIOINTR can be obtained from the APB interface via reading to the read-only 1-bit register GPIOITOP2.
5. When integration tests are run in an integrated system '1' and '0' are written to the GPIOITOP1[7:0] register bits so as to toggle the signal connections between the GPIO and the Interrupt controller.
6. Read from the Interrupt controller's internal registers to verify that the value written into the GPIOITOP1[7:0] register bits is read out through the Interrupt Controller.

NOTE: When the tests are run in an integrated system, the user is expected to replace this read(and the following reads with suitable commands to read the values on these lines through the destination peripherals (Interrupt Controller)

7.5 Integration Test Vector Generation

A ***make Integ*** in the ***integration/bustests*** directory will create ***.tif*** formatted and ***.sim*** formatted vectors in the ***integration/bustest/invec*** directory. This is the default setting. The makefile in ***integration/bustest/invec*** directory can be referred for the make options.

make all can be used with the same result as ***make Integ*** to create a test vector file including all the tests.